



Lattice-based Multi-key Homomorphic Signatures Forward-unforgeable against Signing Key Leakage

Ye Xu and Takashi Nishide

University of Tsukuba, Tsukuba, Japan

Abstract. Homomorphic signature (HS) schemes enable an untrusted server to run some computation over the data signed under the same key and derive a short signature for authenticating the computation result. Fiore et al. (Asiacrypt'16) introduced novel lattice-based *multi-key* homomorphic signatures (MKHS) to support an evaluation of signatures under multiple/different keys, and anyone can verify the resultant signature by using corresponding public verification keys. However, a limitation of their scheme is that even if only one signing key is leaked, a malicious server can forge a signature on a fake computation result involving the inputs of uncorrupted signers. To address this issue, we propose a new scheme built upon the work of Fiore et al., aiming to achieve a stronger security guarantee, which we call *forward unforgeability*, against signing key leakage. Our MKHS scheme is constructed based on the short integer solution (SIS) problem as Fiore et al., and can be forward-unforgeable even if an adversary obtains all the signing keys. Furthermore, we propose a variant by introducing a helper entity to amortize the overhead of signature verifications.

Keywords: Multi-key homomorphic signature · Insider corruption · Lattice

1 Introduction

Outsourcing data and delegating tasks to cloud servers is a growing necessity in modern society. In this model, data is stored and processed on remote servers, with the results made accessible to authorized data users. For example, a hospital may upload medical records to a cloud server, and then a life insurance company can request some statistical results computed from the outsourced medical records on the server. However, cloud environments cannot always guarantee reliable and trustworthy services. To ensure evaluation correctness and data authentication in outsourced computation, the cryptographic primitive known as homomorphic signatures [JMSW02, GVW15] is introduced.

Homomorphic Signatures (HS). Homomorphic signatures allow an evaluator to compute admissible functions on signed data and derive a signature on the computation result. For example, a data owner signs n messages $(m_1, \dots, m_n)$ and obtains signatures $(\sigma_1, \dots, \sigma_n)$, which are then outsourced to a cloud server. By HS, the server (evaluator) can compute a function f on the signed messages and derive a valid signature σ from $(\sigma_1, \dots, \sigma_n)$ for the message $y = f(m_1, \dots, m_n)$. A main property of HS is succinctness, meaning σ 's size is smaller than concatenated $(\sigma_1, \dots, \sigma_n)$. This feature allows a verifier to check the authenticity of computations performed on a large volume of data using only a short piece of information. In real-world scenarios, computations often involve multiple data owners. A trivial way for this situation is to share the same signing key among data owners, which carries a high risk of key leakage. Fiore et al. [FMNP16] extended

E-mail: s2130130@u.tsukuba.ac.jp (Ye Xu), nishide@risk.tsukuba.ac.jp (Takashi Nishide)



the single-key HS scheme of Gorbunov, Vaikuntanathan, and Wichs (GVW) [GVW15] based on the short integer solution (SIS) hardness assumption to the multi-key setting to handle the scenarios where multiple data owners (signers) provide data to the same computation. However, the scheme [FMNP16] did not address the situation called *insider corruption* [LTCW18] in which a signer involved in the computation becomes corrupted later and the corresponding signing key is leaked to the evaluator. As a result, a malicious evaluator holding the signing key can forge a signature regardless of the signed inputs.

Insider Corruption. Informally, insider corruption refers to a situation where at least one signer involved in the computation is corrupted by an adversary (evaluator), leading to the leakage of the signing key. We briefly explain why the lattice-based multi-key homomorphic signature (MKHS) scheme due to Fiore et al. [FMNP16] is forgeable under insider corruption.

Assume that M data owners (signers) with public keys $\{pk_i\}_{i \in [M]}$ outsource N messages $(m_1, \dots, m_N)$ with corresponding signatures $(\sigma_1, \dots, \sigma_N)$ and evaluation keys $\{ek_i\}_{i \in [N]}$ to the cloud server (evaluator) where the public and evaluation keys¹ are made public to potential verifiers. Since their construction is based on the lattice trapdoor function, given $\{ek_i\}_{i \in [N]}$ and any (fake) output y^* , one can easily sample a signature σ^* to pass the verification just with one signing key sk_i (SIS trapdoor of pk_i) corresponding to any $pk_i \in \{pk_i\}_{i \in [M]}$. Hence, a malicious evaluator possessing only one signing key from a corrupted signer can generate a valid signature σ for any message-function pair (y, f) disregarding the inputs from other honest signers. Thus the scheme [FMNP16] is secure only when the computation result does not involve any corrupted signers as already noted in [FMNP16], and also mentioned by Lai et al. [LTCW18, Appx. A].

Lai et al. addressed this issue by utilizing the *strong* zero-knowledge succinct non-interactive argument of knowledge (zk-SNARK) [BG14, FN16] and SNARK in the presence of (signing) oracles (O-SNARK) [FN16]. However, the strong zk-SNARKs based construction requires the assumption of a “strong” proof of knowledge property [BG14, FN16]. Besides, in their O-SNARK based construction, the runtime of the recursive extractor in the security proof can be exponential in the depth of circuits (in the worst case). Moreover, Lai et al. showed that the existence of MKHS schemes under their security model implies succinct non-interactive arguments (SNARGs), whereas it is known that (adaptively sound) SNARGs cannot be based on falsifiable assumptions via black-box reductions [GW11]. This led to a kind of impossibility result for constructing MKHS schemes under insider corruption from standard assumptions, and it was left open to develop MKHS schemes withstanding a weaker yet still reasonable level of security from standard/falsifiable assumptions.

1.1 Our Results

Multi-key Homomorphic Signatures. We extend Fiore et al. [FMNP16] to achieve what we call *forward unforgeability* under insider corruption with lattice-based standard assumptions, despite the aforementioned impossibility result of Lai et al. [LTCW18]. The goal of forward unforgeability is to ensure that, even if all the signing keys are compromised, the adversary cannot produce a forged signature on a computation result that disregards the inputs signed prior to the corruption. This property is useful in applications such as data outsourcing and audit systems, where honestly signed data from the past may be reused in future computations or validations, and it is critical to ensure that such data cannot be tampered with retroactively by the adversary. Forward unforgeability is still meaningful since the results computed from the inputs signed before corruption remain verifiable. For example, consider hospitals outsourcing signed data to a cloud server and

¹The evaluation keys work as identifiers of input messages from the viewpoint of verifiers.

publishing corresponding keys for further verification. During evaluation, the cloud server may corrupt any hospital to obtain a signing key and then return a computation result with the corresponding signature. In our work, any life insurance company can verify the correctness of the result using the already published keys and resultant signature. However, the computation results in [FMNP16] would no longer be reliable under such corruption. We note that the inputs signed after corruption are beyond the scope of forward unforgeability.

More specifically, in our constructions, the forward unforgeability is realized in the following sense. Suppose that an input x is signed by an honest signer id , and the corresponding public/evaluation keys are made public. Then the input x , signed when id is honest, cannot be disregarded in subsequent computations, even if the signer id is later corrupted by a malicious evaluator. Thus, we construct the first MKHS scheme achieving forward unforgeability under insider corruption with standard assumptions, thereby solving the open problem posed in [LTWC18]. In our security model, all the signers are initially honest, with their key pairs/signatures generated by the signing oracle as in [FMNP16]. When a signer is corrupted later, the corresponding signing keys are leaked to the adversary. The goal of the adversary (evaluator with leaked signing keys) is to break forward unforgeability, i.e., to forge a signature on a computation result that does not respect (part of) the inputs where all the public/evaluation keys originate from the keys generated when the corresponding signers were honest.

In the security model of Lai et al. [LTWC18], the key pairs/signatures of honest signers are generated by the signing oracle while the key pairs/signatures of corrupted signers are generated by the adversary. The goal of the adversary is to forge a signature on the computation result that does not respect the inputs of honest signers where honest signers stay uncorrupted during the security game (i.e., no adaptive corruption queries from the adversary to capture forward unforgeability in the security game of [LTWC18]²). Since our security model captures forward unforgeability where the key pairs/signatures of the corrupted signers are generated by the signing oracle, it is insufficient to imply the existence of SNARGs, thus circumventing the impossibility result in [LTWC18]. Notably, our MKHS schemes based on standard assumptions remain unforgeable even with all signing keys compromised, achieving a level of security beyond what was possible in Fiore et al. [FMNP16].

A Variant with Amortized Verification Efficiency. Our second construction enhances the efficiency of our basic scheme in multi-dataset scenarios while retaining similar security guarantees. The efficiency is defined in an amortized sense. A feature of this variant is the reusability of evaluation keys across different datasets. To this end, we introduce a helper entity (called proxy server) inspired by the trusted authority presented in the model of Peikert et al. [PPS21]. This design allows verifiers to preprocess the evaluation keys before receiving the signatures, thus streamlining the verification of the same program over different datasets. This entity has its own key pair, and we assume this entity stays uncorrupted.

Moreover, to reduce the level of trust required in this entity, we devise an interactive signing algorithm that allows a signer to keep the messages to be signed hidden from the helper entity. To achieve this, we develop a strategy to handle corruption queries for insider corruption and the simulation of our interactive signing algorithm in the security proofs.

Finally, we present a further extended variant that supports multiple proxy servers and remains forward-unforgeable as long as at least one proxy server is honest.

²It appears that achieving a form of forward unforgeability in [LTWC18] requires further adjustments.

1.2 Technical Overview

Here we present an overview of our forward unforgeable multi-key homomorphic signature schemes from lattices. The details are in §4 and 5, respectively.

Basic Multi-key Homomorphic Signatures. MKHS is built on the model of labeled programs and data of Gennaro and Wichs [GW13]. A labeled program $\mathcal{P}$ is a tuple $(f, \ell_1, \dots, \ell_t)$ where $f: \mathcal{X}^t \rightarrow \mathcal{X}$ is a function of t variables and $\ell_i \in \{0, 1\}^*$ is a label for the i -th input of f . Every data item x_i is assigned a unique label ℓ_i , which can serve as an identifier for the outsourced data. Our starting point is Fiore et al.’s MKHS scheme [FMNP16] based on the standard assumption, the short integer solution (SIS) problem over standard lattices. We briefly recall the construction of [FMNP16] and its limitation under insider corruption.

In the notion of *multi-key homomorphic signatures*, one aims to verify the outputs of computations executed on data signed under different keys belonging to different clients/signers. We assume that each signer has an identity id and the signer’s keys are associated with id . To distinguish data from different signers, each data item is assigned a label $\ell := (id, \tau)$, where id represents the signer’s identity and τ is a tag. In the context of MKHS, multiple data items (e.g., $x_1, \dots, x_t$) from multiple signers, when grouped into a set, are often referred to as a *dataset*, and the computation is assumed to be executed only on data within the same dataset. In the definitions, such a dataset is identified by an arbitrary string Δ .

Signer id_i generates a uniformly random SIS matrix $\mathbf{A}_{id_i} \in \mathbb{Z}_q^{n \times m}$ along with a secret trapdoor as in [MP12], which are set as a public/signing key pair (pk, sk) . To sign a data item x_ℓ , a uniformly random matrix $\mathbf{V}_\ell \in \mathbb{Z}_q^{n \times m}$ is set and published as a public parameter, serving as an evaluation key in the construction. Using the trapdoor, the signer can efficiently sample a short (i.e., having small norm) preimage $\mathbf{U}_\ell \in \mathbb{Z}_q^{m \times m}$ such that $\mathbf{A}_{id_i} \mathbf{U}_\ell + x_\ell \cdot \mathbf{G} = \mathbf{V}_\ell$, where $\mathbf{G}$ is the gadget matrix. The short matrix $\mathbf{U}_\ell$ is viewed as the signature of x_ℓ . We note that by the SIS assumption, it is hard to find a short matrix $\mathbf{U}' \in \mathbb{Z}_q^{m \times m}$ such that $\mathbf{A}_{id_i} \mathbf{U}' + x_\ell \cdot \mathbf{G} = \mathbf{V}_\ell$ holds.

The construction of Fiore et al. [FMNP16] works as follows. Assume that a program $\mathcal{P}$ is evaluated on a dataset $(x_1, \dots, x_t)$ and that each x_i is given by id_i for simplicity. Each signer $id_i (\in \{id_1, \dots, id_t\})$ generates a uniformly random SIS matrix $\mathbf{A}_{id_i} \in \mathbb{Z}_q^{n \times m}$ along with a secret trapdoor $\mathbf{T}_{id_i}$ as in [MP12], which are set as a public/signing key pair $(pk_i, sk_i) := (\mathbf{A}_{id_i}, \mathbf{T}_{id_i})$. A dataset $(x_1, \dots, x_t)$ along with the signatures $(\mathbf{U}_1, \dots, \mathbf{U}_t)$ are the inputs of a program $\mathcal{P}$, which can be treated as an arithmetic circuit over $\mathbb{Z}_2$ consisting of fan-in-2 addition and multiplication gates. For ease of exposition, $\mathbf{U}_{\ell=(id, \tau)}$ is denoted as $\mathbf{U}_{id}$ here, and the signatures are evaluated according to the identities³. We note that, in [FMNP16], the number of evaluation keys is the same as the number of data items in a dataset. Finally, the server returns the computation result $y = f(x_1, \dots, x_t)$ along with its resultant signature $\mathbf{U}_y := (\mathbf{U}_{id_1}, \dots, \mathbf{U}_{id_t})$. To verify the result, one (the verifier) needs to homomorphically evaluate f on the evaluation keys $(\mathbf{V}_1, \dots, \mathbf{V}_t)$ resulting in some $\mathbf{V}_f$, and then checks if $\sum_{id \in I} \mathbf{A}_{id} \mathbf{U}_{id} + y \cdot \mathbf{G} = \mathbf{V}_f$ holds, where I denotes the set of identities involved in the computation of f (e.g., $I = (id_1, \dots, id_t)$). If no corruption occurs, (i.e., no signing key is leaked to the server), then given $\mathbf{V}_f$, it is hard for the server to find a short $\mathbf{U}_y^* = \{\mathbf{U}_{id}^*\}_{id \in I}$ that satisfies the verification equation due to the SIS hardness assumption. However, if even only one signer is corrupted by the server and leaks the signing key, it allows the server to forge the signature for an incorrect result⁴, disregarding

³When the signer provides multiple data items, the signatures are indexed by the corresponding labels.

⁴To forge a signature for an incorrect result y^* , the malicious server holding one leaked signing key can proceed as follows. Assume that the signer $id_c \in I$ is corrupted by the server. For all uncorrupted signers $id \in I \setminus \{id_c\}$, the server randomly selects short $\mathbf{U}_{id}^*$ ’s and computes $\mathbf{V}_c = \mathbf{V}_f - \sum_{id \in I \setminus \{id_c\}} \mathbf{A}_{id} \mathbf{U}_{id}^*$. The server then generates $\mathbf{U}_{id_c}^*$ using the leaked signing key such that $\mathbf{A}_{id_c} \mathbf{U}_{id_c}^* + y^* \cdot \mathbf{G} = \mathbf{V}_c$. Thus, the

the original inputs. Therefore, the main challenge of this work is to prevent the malicious server from producing a signature on an incorrect computation result that disregards the previously signed inputs, and this property is defined as *forward unforgeability* against insider corruption.

In the security game of our forward unforgeability, all the signers are initially honest and their key pairs/signatures are generated by the signing oracle. When any signer is later corrupted, the adversary receives the corresponding signing key. Forward unforgeability guarantees that even if the adversary steals all the signing keys, the past signatures produced under those signing keys when the signers are uncorrupted cannot be invalidated in the context of insider corruption.

To address the aforementioned issue and achieve forward unforgeability from falsifiable assumption, we introduce a uniformly random public matrix $\mathbf{M}$ and change the way of generating the evaluation keys. Specifically, to sign the input x_ℓ , each signer first generates the evaluation key as $\mathbf{V}_\ell := \mathbf{M}\mathbf{W}_\ell + x_\ell \cdot \mathbf{G}$ where $\mathbf{W}_\ell \in \mathbb{Z}_q^{m \times m}$ is a uniformly random short matrix sampled by the signer. Since $\mathbf{M}$ is a random matrix, its “trapdoor” is unknown to anyone. The signer signs the input x_ℓ with the signing key to obtain a short $\mathbf{U}_\ell$ such that $\mathbf{A}_{id}\mathbf{U}_\ell + x_\ell \cdot \mathbf{G} = \mathbf{V}_\ell$. The signature on the input x_ℓ consists of both $\mathbf{W}_\ell$ and $\mathbf{U}_\ell$. During the evaluation, $\mathbf{W}_\ell$ ’s can be evaluated as the signatures from a “virtual signer” whose public key is $\mathbf{M}$. After the evaluation of program $\mathcal{P}$, the server outputs the final signature denoted as $\sigma_y := (\{\mathbf{U}_{id}\}_{id \in I}, \mathbf{W})$. In the verification step, in addition to checking if $\sum_{id \in I} \mathbf{A}_{id}\mathbf{U}_{id} + y \cdot \mathbf{G} = \mathbf{V}_f$, a verifier also checks if $\mathbf{M}\mathbf{W} + y \cdot \mathbf{G} = \mathbf{V}_f$ holds. These two equations ensure the authenticity of inputs and the correctness of computation results.

A Variant for Multiple Datasets. When the same program $\mathcal{P}$ is executed on different datasets, verification efficiency can be enhanced by reusing evaluation keys $\{\mathbf{V}_\ell\}$ across multiple datasets. Specifically, the verification step can be split into an offline phase and an online phase. The evaluation keys are evaluated in the offline phase only once to get $\mathbf{V}_f$, which can then be used repeatedly in the online phase of verification for the same program (of function f) executed on different datasets. To realize this, we propose a variant by introducing an uncorrupted proxy server that works like the trusted authority in Peikert et al. [PPS21], achieving amortized verification efficiency in the multi-dataset scenario.

In this variant, the evaluation keys $\{\mathbf{V}_\ell\}$ are uniformly sampled and set as public parameters common across multiple datasets, while the signers and proxy server generate fresh public/signing key pairs $(\mathbf{A}_{id}^\Delta, \mathbf{T}_{id}^\Delta)$, $(\mathbf{A}_{ser}^\Delta, \mathbf{T}_{ser}^\Delta)$ respectively for each dataset with identifier Δ as in [FMNP16]. The public key $\mathbf{A}_{ser}^\Delta$ of the proxy server serves as the public matrix $\mathbf{M}$ in the basic scheme for each dataset with identifier Δ . Similarly to the basic scheme, the signer id ’s signature on input x_ℓ (in the dataset with identifier Δ) contains both $\mathbf{W}_\ell$ and $\mathbf{U}_\ell$ satisfying $\mathbf{V}_\ell = \mathbf{A}_{ser}^\Delta \mathbf{W}_\ell + x_\ell \mathbf{G} = \mathbf{A}_{id}^\Delta \mathbf{U}_\ell + x_\ell \mathbf{G}$ where the signer computes $\mathbf{U}_\ell$ as in the basic scheme, while the component $\mathbf{W}_\ell$ is viewed as the signature from the proxy server. However, instead of letting the proxy server directly access the input x_ℓ and generate the signature $\mathbf{W}_\ell$, we devise an interactive signing algorithm between a signer and the proxy server to reduce the level of trust required in the proxy server. To this end, the signer chooses a random short $\mathbf{W}_{\ell, \text{temp}}$, provides, to the proxy server, temporary value $\mathbf{V}_{\ell, \text{temp}}$ satisfying $\mathbf{V}_{\ell, \text{temp}} = \mathbf{A}_{ser}^\Delta \cdot \mathbf{W}_{\ell, \text{temp}} + x_\ell \cdot \mathbf{G}$, and obtains temporary $\mathbf{W}_{s, \text{temp}}$ satisfying $\mathbf{V}_\ell - \mathbf{V}_{\ell, \text{temp}} = \mathbf{A}_{ser}^\Delta \mathbf{W}_{s, \text{temp}}$ from the proxy server. As a result, the signer can compute short $\mathbf{W}_\ell = \mathbf{W}_{s, \text{temp}} + \mathbf{W}_{\ell, \text{temp}}$ satisfying $\mathbf{V}_\ell = \mathbf{A}_{ser}^\Delta \mathbf{W}_\ell + x_\ell \mathbf{G}$ without providing the input x_ℓ directly to the proxy server. To simulate the interactive signing process in the security proof, we develop a strategy in which we modify how to generate evaluation keys and signatures of the proxy server.

Finally, to mitigate a trust assumption on a single proxy server, we show that our

server can produce a forged signature $\mathbf{U}_y^* = \{\mathbf{U}_{id}^*\}_{id \in I}$ such that $\sum_{id \in I} \mathbf{A}_{id} \mathbf{U}_{id}^* + y^* \cdot \mathbf{G} = \mathbf{V}_f$, which satisfies the verification equation.

scheme can be extended to support multiple independent proxy servers, ensuring forward unforgeability as long as one proxy server remains honest.

1.3 Related Work

Homomorphic signatures (HS) were initially designed for linear functions and applied in network coding [ALP13, BFKW09, CFW12]. Gorbunov et al. [GVW15] introduced the first (leveled) fully homomorphic signature (FHS) scheme for arbitrary circuits of bounded polynomial depth. Later, Fiore et al. [FMNP16] extended this to the multi-key setting, formalizing multi-key homomorphic signatures (MKHS) and presenting the first MKHS scheme. However, their scheme did not address insider corruption, making signatures forgeable if any involved signer is corrupted. Based on the security model of [FMNP16], two MKHS schemes were proposed for linear functions [AP19, SBB19], but both schemes remained insecure under insider corruption similarly. Lai et al. [LTCW18] addressed insider corruption with two constructions based on zk-SNARKs, which relied on non-falsifiable assumptions. Balbás et al. [BCFL23] proposed a novel chainable functional commitment (FC) scheme and used it to realize single-key homomorphic signatures. Recently, Anthoine et al. [ABF24] combined FC and batch arguments for NP (BARGs) to realize a novel succinct MKHS. However, their security model does not allow the adversary to include the inputs of corrupted signers in the forgery, still adhering to a security model similar to that of [FMNP16] in terms of insider corruption. Thus, no existing MKHS schemes based on falsifiable assumptions provide a level of security against insider corruption.

2 Preliminaries

For positive integer $q \geq 2$, we use $\mathbb{Z}_q$ to denote the integers in $(-\frac{q}{2}, \frac{q}{2}]$. For $Q \in \mathbb{N}$, we refer to $[Q]$ as $[Q] := \{1, 2, \dots, Q\}$. We use bold upper-case letters (e.g., $\mathbf{A}$) and bold lower-case letters (e.g., $\mathbf{a}$) to denote matrices and vectors, respectively. For a distribution $\mathcal{D}$, we use the notation $d \xleftarrow{\$} \mathcal{D}$ to represent the process of sampling a random value d from $\mathcal{D}$. For a set X , we use $x \xleftarrow{\$} X$ to denote the process of uniformly choosing a random value x from X . We let λ denote the security parameter and $\text{negl}(\lambda)$ denote a negligible function $f(\lambda)$, if $f(\lambda) = O(\lambda^{-c})$ for any constant $c > 0$. For a vector $\mathbf{u} := (u_1, \dots, u_n) \in \mathbb{Z}_q^n$, we write $\|\mathbf{u}\|_\infty \leq \beta$ if each u_i satisfies $|u_i| \leq \beta$. Similarly, for a matrix $\mathbf{U} := [u_{i,j}] \in \mathbb{Z}_q^{n \times m}$, we write $\|\mathbf{U}\|_\infty \leq \beta$ if each $u_{i,j}$ satisfies $|u_{i,j}| \leq \beta$.

Entropy and Statistical Distance. For two random variables X, Y with support $\mathcal{X}, \mathcal{Y}$ respectively, the statistical distance is defined as

$$\mathbf{SD}(X, Y) := \frac{1}{2} \sum_{u \in \mathcal{X} \cup \mathcal{Y}} |\Pr[X = u] - \Pr[Y = u]|.$$

We say that X and Y are statistically close, denoted by $X \stackrel{\text{stat}}{\approx} Y$, if $\mathbf{SD}(X, Y) = \text{negl}(\lambda)$. The *min-entropy* of a random variable X is defined as

$$\mathbf{H}_\infty(X) := -\log(\max_x \Pr[X = x]).$$

The *(average-)conditional min-entropy* of a random variable X conditioned on a correlated variable Y is defined as

$$\mathbf{H}_\infty(X|Y) := -\log(\mathbf{E}_{y \leftarrow Y}[\max_x \Pr[X = x|Y = y]]).$$

The optimal probability of an unbounded adversary that can guess X , given the correlated value Y , is $2^{-\mathbf{H}_\infty(X|Y)}$.

Lemma 1 ([DORS08]). *Let X, Y be arbitrary random variables where the support of Y lies in $\mathcal{Y}$. Then $\mathbf{H}_\infty(X|Y) \geq \mathbf{H}_\infty(X) - \log(|\mathcal{Y}|)$.*

Lattices. Let n, m, q be positive integers. The q -ary integer lattices defined by a random matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ is $\Lambda_q^\perp(\mathbf{A}) = \{\mathbf{u} \in \mathbb{Z}^m : \mathbf{A} \cdot \mathbf{u} = \mathbf{0} \bmod q\}$. For a vector $\mathbf{v} \in \mathbb{Z}_q^n$, the coset of $\Lambda_q^\perp(\mathbf{A})$ is $\Lambda_q^\mathbf{v}(\mathbf{A}) = \{\mathbf{u} \in \mathbb{Z}^m : \mathbf{A} \cdot \mathbf{u} = \mathbf{v} \bmod q\}$.

Short Integer Solution (SIS) Problem. Let n, m, q, β be integers. In $\text{SIS}(n, m, q, \beta)$ problem, given a random matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$, adversary $\mathcal{A}$ is required to find a nonzero vector $\mathbf{u} \in \mathbb{Z}_q^m$ such that $\|\mathbf{u}\|_\infty \leq \beta$ and $\mathbf{A} \cdot \mathbf{u} = \mathbf{0} \bmod q$.

Definition 1 (SIS [MR04]). For parameters $n = n(\lambda)$, $m = m(\lambda)$, $q = q(\lambda)$, $\beta = \beta(\lambda)$ defined in terms of security parameter λ , the $\text{SIS}(n, m, q, \beta)$ hardness assumption holds if for any PPT adversary $\mathcal{A}$, we have

$$\Pr[\mathbf{A} \cdot \mathbf{u} = \mathbf{0} \wedge \mathbf{u} \neq \mathbf{0} \wedge \|\mathbf{u}\|_\infty \leq \beta(\lambda) : \mathbf{A} \xleftarrow{\$} \mathbb{Z}_q^{n \times m}, \mathbf{u} \leftarrow \mathcal{A}(1^\lambda, \mathbf{A})] \leq \text{negl}(\lambda).$$

The SIS problem is known to be as hard as certain worst-case problems such as SIVP in standard lattices [MR04]. In this work, we follow the choice of parameters in Fiore et al. [FMNP16] to assure the hardness. We assume that for any $\beta = 2^{\text{poly}(\lambda)}$, there are some $n = \text{poly}(\lambda)$, $q = 2^{\text{poly}(\lambda)} > \beta$ such that for all $m = \text{poly}(\lambda)$, the $\text{SIS}(n, m, q, \beta)$ hardness assumption holds.

Trapdoors for Lattices. It will be easy to solve the SIS problem by sampling a random matrix $\mathbf{A}$ with a trapdoor $\mathbf{T}$. We recall the summarized lemma given in [GVW15, FMNP16] for the known results about the trapdoor.

Lemma 2 ([GPV08, AP11, MP12]). *Let integers $n \geq 1$ and $q \geq 2$. There exist some $m^* = m^*(n, q) = O(n \log q)$ and $\beta_{\text{Sam}} = \beta_{\text{Sam}}(n, q) = O(n\sqrt{\log q})$. For all $m \geq m^*$ and all k (polynomial in n), there exist efficient algorithms TrapGen , SamPre , Sam with the properties below:*

1. $\mathbf{U} \leftarrow \text{Sam}(1^m, 1^k, q)$ outputs a random sample $\mathbf{U} \in \mathbb{Z}_q^{m \times k}$ which satisfies $\|\mathbf{U}\|_\infty \leq \beta_{\text{Sam}}$ (with probability 1).
2. $(\mathbf{A}, \mathbf{T}) \leftarrow \text{TrapGen}(1^n, 1^m, q)$ outputs a full-rank matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ with trapdoor $\mathbf{T}$ such that $\mathbf{A}$ is statistically close to uniform over $\mathbb{Z}_q^{n \times m}$. For $\mathbf{V}' \xleftarrow{\$} \mathbb{Z}_q^{n \times k}$, $\mathbf{U}' \leftarrow \text{SamPre}(\mathbf{A}, \mathbf{V}', \mathbf{T})$ samples the preimage of $\mathbf{V}'$ and satisfies $\mathbf{V}' = \mathbf{A}\mathbf{U}'$ and $\|\mathbf{U}'\|_\infty \leq \beta_{\text{Sam}}$. For $\mathbf{A}' \xleftarrow{\$} \mathbb{Z}_q^{n \times m}$, $\mathbf{U} \leftarrow \text{Sam}(1^m, 1^k, q)$, $\mathbf{V} := \mathbf{A}\mathbf{U}$ and $\mathbf{A}, \mathbf{T}, \mathbf{V}', \mathbf{U}'$, we have the statistical indistinguishability (negligible in n): $\mathbf{A} \stackrel{\text{stat}}{\approx} \mathbf{A}'$ and $(\mathbf{A}, \mathbf{T}, \mathbf{U}, \mathbf{V}) \stackrel{\text{stat}}{\approx} (\mathbf{A}', \mathbf{T}, \mathbf{U}', \mathbf{V}')$.
3. Given parameters n, m, q , there exists an efficiently and deterministically computable matrix $\mathbf{G} \in \mathbb{Z}_q^{n \times m}$ and a deterministic polynomial-time algorithm $\mathbf{G}^{-1}$. $\mathbf{G}^{-1}$ takes the input $\mathbf{V}^{(n \times k)}$ for any integer k and outputs $\mathbf{R} = \mathbf{G}^{-1}(\mathbf{V})$ such that $\mathbf{R} \in \{0, 1\}^{m \times k}$ and $\mathbf{G} \cdot \mathbf{R} = \mathbf{V}$.

3 Definitions and Security Model

3.1 Formalizing Multi-Key Homomorphic Signatures (MKHS)

Notation. To specify the authenticated data and evaluate the program, the scheme relies on the notion of labeled data and programs [GW13]. Each input bit of a labeled

program $\mathcal{P}$ is associated with a label ℓ . A labeled program $\mathcal{P}$ is a tuple $(f, \ell_1, \dots, \ell_t)$, where $f: \mathcal{X}^t \rightarrow \mathcal{X}$ is a circuit with t input labels $\ell_{i \in [t]} \in \{0, 1\}^*$ for t input wires. The set of these t inputs (i.e., data items provided by multiple signers) is often referred to as a dataset identified by $\Delta \in \{0, 1\}^*$. A composed program $\mathcal{P}^* = g(\mathcal{P}_1, \dots, \mathcal{P}_k)$ is formed by evaluating a function $g: \mathcal{X}^k \rightarrow \mathcal{X}$ on the outputs of programs $\mathcal{P}_1, \dots, \mathcal{P}_k$. In MKHS, every signer has an identity id in the identity space $\mathcal{ID}$, and a label for a data item is $\ell := (id, \tau)$, with τ being a tag for the item.

Definition 2 (MKHS). A general *MKHS* scheme consists of the following five polynomial-time algorithms (**Setup**, **KeyGen**, **Sign**, **Eval**, **Ver**) specified as follows:

Setup(1^λ). On input a security parameter λ , the setup algorithm outputs public parameters $\mathbf{pp}$, which contains a set of parameters $\mathbf{prms}$ involved in the scheme, the identity space $\mathcal{ID}$, the message space $\mathcal{X}$, the tag space $\mathcal{T}$ and the label space $\mathcal{L} := \mathcal{ID} \times \mathcal{T}$. We use $id \in \mathcal{P}$ to denote the identities involved in the labeled program $\mathcal{P}$.

KeyGen($\mathbf{pp}, \Delta, (\ell, x)^5$). The key generation algorithm consists of two sub-algorithms:

1. Signing-public key generation: **KeyGen**₁($\mathbf{pp}, \Delta$) takes as input the public parameter $\mathbf{pp}$ and dataset identifier Δ , then outputs the key pair (sk, pk) . The key pairs are freshly generated for each dataset.
2. Evaluation key generation: **KeyGen**₂ works differently depending on the setting:
 - Single-dataset setting: the algorithm **KeyGen**₂($\mathbf{pp}, \Delta, (\ell, x)$) takes as input the public parameter $\mathbf{pp}$, a dataset identifier Δ , a message x with its label $\ell := (id, \tau)$, then outputs a message-specific evaluation key ek_ℓ .
 - Multi-dataset setting: the algorithm **KeyGen**₂($\mathbf{pp}$) takes as input the public parameter $\mathbf{pp}$ and outputs the set of evaluation keys $ek (= \{ek_i\}_{i \in [t]})$. We note that the set of evaluation keys ek can be reused across multiple datasets.

The key triple is associated with the identity $id \in \mathcal{ID}$ and can be denoted as $(sk_{id}, pk_{id}, ek_{id})$.

Sign($sk, pk, ek, \Delta, \ell, x$). The signing algorithm takes as input a signing key sk , a public key pk , an evaluation key ek , a dataset identifier Δ , a message x with its label ℓ and outputs a signature σ .

Eval($f, \{x_i, \sigma_i, ek_i\}_{i \in [t]}$). The evaluation algorithm takes as input a function $f: \mathcal{X}^t \rightarrow \mathcal{X}$ with a set of messages, signatures and evaluation keys $\{x_i, \sigma_i, ek_i\}_{i \in [t]}$. It outputs computation result y and evaluated signature σ .

Ver($\mathcal{P}, \Delta, \{pk_{id}, ek_{id}\}_{id \in \mathcal{P}}, y, \sigma$). The verification algorithm takes as input a labeled program $\mathcal{P} := (f, \ell_1, \dots, \ell_t)$, a dataset identifier Δ , a set of involved public and evaluation keys $\{pk_{id}, ek_{id}\}_{id \in \mathcal{P}}$, a message y , and a homomorphic signature σ . The algorithm outputs 0 (reject) or 1 (accept).

Signing Correctness. An MKHS scheme satisfies signing correctness if for any $\mathbf{pp} \leftarrow \mathbf{Setup}(1^\lambda)$, any $(sk, pk, ek) \leftarrow \mathbf{KeyGen}(\mathbf{pp}, \Delta)$, any label $\ell = (id, \tau) \in \mathcal{L}$, any message $x \in \mathcal{X}$, any dataset identifier $\Delta \in \{0, 1\}^*$, and any signature $\sigma \leftarrow \mathbf{Sign}(sk, pk, ek, \Delta, \ell, x)$, we have $\mathbf{Ver}(\mathcal{I}_\ell, \Delta, pk, ek, x, \sigma) = 1$ with all but negligible probability, where $\mathcal{I}_\ell$ is the identity program over the dataset Δ .

Evaluation Correctness. An MKHS scheme satisfies evaluation correctness if, for any public parameters $\mathbf{pp} \leftarrow \mathbf{Setup}(1^\lambda)$, any set of key triples $\{(sk_{id}, pk_{id}, ek_{id})\}_{id \in \widetilde{\mathcal{ID}}} \leftarrow$

⁵(ℓ, x) is only required in the single-dataset setting.

KeyGen(pp, Δ) for some $\widetilde{\mathcal{ID}} \subset \mathcal{ID}$, any dataset identifier $\Delta \in \{0, 1\}^*$, any function $g: \mathcal{X}^t \rightarrow \mathcal{X}$, and any set of program/message/signature triples $\{(\mathcal{P}_i, x_i, \sigma_i)\}_{i \in [t]}$ such that $\text{Ver}(\mathcal{P}_i, \Delta, \{pk_{id}, ek_{id}\}_{id \in \mathcal{P}_i}, x_i, \sigma_i) = 1$ for all $i \in [t]$, it holds that $\text{Ver}(\mathcal{P}', \Delta, \{pk_{id}, ek_{id}\}_{id \in \mathcal{P}'}, y, \sigma') = 1$ with all but negligible probability, where $y = g(x_1, \dots, x_t)$, $\mathcal{P}' = g(\mathcal{P}_1, \dots, \mathcal{P}_t)$, and $\sigma' = \text{Eval}(g, \{x_i, \sigma_i, ek_i\}_{i \in [t]})$.

Succinctness. Let $\text{pp} \leftarrow \text{Setup}(1^\lambda)$, $\mathcal{P} = (f, \ell_1, \dots, \ell_t)$ with $\ell_i = (id_i, \tau_i)$, $\{(sk_{id_i}, pk_{id_i}, ek_{id_i})\}_{id_i \in \mathcal{P}} \leftarrow \text{KeyGen}(\text{pp}, \Delta)$, $\sigma_i \leftarrow \text{Sign}(sk_{id_i}, pk_{id_i}, ek_{id_i}, \Delta, \ell_i, x_i)$ for all $i \in [t]$. An MKHS scheme is succinct if there is a fixed polynomial p such that $|\sigma| = p(\lambda, n, \log t)$, where $\sigma = \text{Eval}(f, \{x_i, \sigma_i, ek_i\}_{i \in [t]})$ and $n = |\{id \in \mathcal{P}\}|$.

Efficient Verification. An MKHS scheme satisfies efficient verification if the verification algorithm can be split into *offline* and *online phases* [CFW14]. In an *offline phase*, $ek_{\mathcal{P}}$ is precomputed from the given evaluation keys $\{ek_{id}\}_{id \in \mathcal{P}}$ and a labeled program $\mathcal{P}$. $ek_{\mathcal{P}}$ can then be used in the *online phase* to verify the signatures w.r.t. $\mathcal{P}$ and any dataset Δ . Crucially, $ek_{\mathcal{P}}$ can be reused an unbounded number of times, and the cost of *online phase* is much cheaper than running the program. The formal definition is as follows.

Definition 3 (Efficient Verification [CFW14]). An MKHS scheme for multi-labeled programs satisfies efficient verification if there exist two additional algorithms (**VerPrep**, **EffVer**) such that:

VerPrep($\mathcal{P}, \{ek_{id}\}_{id \in \mathcal{P}}$): Given the labeled program $\mathcal{P} = (f, \ell_1, \dots, \ell_t)$ and a set of evaluation keys $\{ek_{id}\}_{id \in \mathcal{P}}$, the precomputation algorithm outputs $ek_{\mathcal{P}}$. This is independent of the dataset identifier Δ .

EffVer($ek_{\mathcal{P}}, \{pk_{id}\}_{id \in \mathcal{P}}, \Delta, x, \sigma$): Given $ek_{\mathcal{P}}$, a set of public keys $\{pk_{id}\}_{id \in \mathcal{P}}$, a dataset identifier Δ , a message $x \in \mathcal{X}$, and a signature σ , the efficient verification algorithm outputs 0 (reject) or 1 (accept).

The above algorithms are required to satisfy the following two properties:

Correctness. Let $\{(sk_{id}, pk_{id}, ek_{id})\}_{id \in \mathcal{ID}}$ be a set of honestly generated keys and $(\mathcal{P}, \Delta, x, \sigma)$ be a tuple such that $\text{Ver}(\mathcal{P}, \Delta, \{pk_{id}, ek_{id}\}_{id \in \mathcal{P}}, x, \sigma) = 1$. Then for every $ek_{\mathcal{P}} \leftarrow \text{VerPrep}(\mathcal{P}, \{ek_{id}\}_{id \in \mathcal{P}})$, $\text{EffVer}(ek_{\mathcal{P}}, \{pk_{id}\}_{id \in \mathcal{P}}, \Delta, x, \sigma) = 1$ holds with overwhelming probability.

Amortized Efficiency. Let $\mathcal{P}$ be a program, let $(x_1, \dots, x_t) \in \mathcal{X}^t$ be a set of inputs and let $s(t)$ be the time required to compute $\mathcal{P}(x_1, \dots, x_t)$ with output x . If $ek_{\mathcal{P}} \leftarrow \text{VerPrep}(\mathcal{P}, \{ek_{id}\}_{id \in \mathcal{P}})$, then the time required for computing $\text{EffVer}(ek_{\mathcal{P}}, \{pk_{id}\}_{id \in \mathcal{P}}, \Delta, x, \sigma)$ is $s' = o(s(t))$.

3.2 Security Models

Our security model allows the adversary to corrupt all clients (signers) involved in the computation. We consider a selective security game for our MKHS, where the adversary declares the message-tag pairs before seeing the keys. We note that the selective construction can be converted into an adaptive construction using the transformation in [FMNP16], so we omit the details of transformation in this work. We define the security via the following game $\text{Game}_{\mathcal{A}, \text{MKHS}}^{\text{HomUF-CMA}}(\lambda)$ between an adversary $\mathcal{A}$ and a challenger $\mathcal{C}$. $\mathcal{A}$ is allowed to corrupt all the signers involved in the computation to obtain the signing keys during the corruption query phase.

Init. $\mathcal{A}$ sends to $\mathcal{C}$ a set of message-tag pairs $\{(x_\tau, \tau)\}_{\tau \in \mathcal{T}_{i,k} \subseteq \mathcal{T}}$ for each $i \in [Q_{id}]$ and $k \in [Q_\Delta]$, where $\mathcal{T} := \{\tau_1, \dots, \tau_T\}$, Q_{id} and Q_Δ are the total numbers of identities and datasets that will be queried during the game as in [FMNP16]. The indices i, k denote the i -th freshly queried identity and the k -th freshly queried dataset.

During the simulation, $\mathcal{C}$ holds ordered lists of these queries. For clarity, we omit this detail in the description of the game. We note that $\mathcal{A}$ declares in advance all the messages and tags that will be queried during the game without needing to specify the identity/dataset names.

Setup. $\mathcal{C}$ runs $\text{pp} \leftarrow \text{Setup}(1^\lambda)$ and sends pp to $\mathcal{A}$.

Signing Query. $\mathcal{A}$ can adaptively query (id, Δ) to $\mathcal{C}$, where id is an identity and Δ is a dataset identifier. If this is the first query for the dataset Δ , $\mathcal{C}$ initializes a list $L_\Delta = \emptyset$. If this is the first signing query for id in the query of Δ , $\mathcal{C}$ generates new keys $(sk_{id,\Delta}, pk_{id,\Delta}, ek_{id})$ for id . For all labels $\ell := (id, \tau)_{\tau \in \mathcal{T}_{i,k}}$ of declared messages $\{x_\tau\}_{\tau \in \mathcal{T}_{i,k}}$ associated with id , $\mathcal{C}$ generates corresponding signatures $\{\sigma_\tau\}_{\tau \in \mathcal{T}_{i,k}}$ and appends (ℓ, x_τ) to L_Δ . Finally, $\mathcal{C}$ sends to $\mathcal{A}$ the key pair $(pk_{id,\Delta}, ek_{id})$ and the signatures $\{\sigma_\tau\}_{\tau \in \mathcal{T}_{i,k}}$. If $\mathcal{A}$ has queried (id, Δ) before, $\mathcal{C}$ ignores the query.

Corruption Query. The adversary has access to a corruption oracle. At the beginning of the game, $\mathcal{C}$ initializes an empty list $L_{\text{cor}} = \emptyset$. There exist two situations for a corruption query:

1. If $\mathcal{A}$ corrupts id (where id is the i -th queried identity) after sending (possibly several) signing queries, it sends $\mathcal{C}$ the corruption query for id with remaining unqueried dataset identifiers. Using the dataset identifiers and declared message-tag pairs, $\mathcal{C}$ then generates new key triple $(sk_{id,\Delta}, pk_{id,\Delta}, ek_{id})$ of id for each unqueried dataset Δ , generates signatures for declared but unqueried messages associated with id , and stores (ℓ, x_τ) in a list L_Δ according to the dataset identifiers. Finally, $\mathcal{C}$ replies to $\mathcal{A}$ with all the keys assigned to id and all the signatures $\{\sigma_\tau\}_{\tau \in \mathcal{T}_{i,\cdot}}$ associated with id , and updates $L_{\text{cor}} \leftarrow L_{\text{cor}} \cup id$. If $id \in L_{\text{cor}}$, $\mathcal{C}$ ignores the query.
2. If $\mathcal{A}$ sends a corruption query for id along with any remaining unqueried dataset identifiers before sending any signing query for id , i.e., id has not been queried in the signing oracle, $\mathcal{C}$ interprets this as the scenario where $\mathcal{A}$ sends the signing queries followed by the corruption query⁶. $\mathcal{C}$ then generates key triple $(sk_{id,\Delta}, pk_{id,\Delta}, ek_{id})$ for id for each dataset Δ , generates signatures for all the declared message-tag pairs tied to id , and stores all (ℓ, x_τ) associated with id in the list L_Δ of each dataset Δ . Then $\mathcal{C}$ updates $L_{\text{cor}} \leftarrow L_{\text{cor}} \cup id$. Finally, $\mathcal{C}$ replies to $\mathcal{A}$ with all the keys and signatures associated with id . If $id \in L_{\text{cor}}$, $\mathcal{C}$ ignores the query.

Forgery. $\mathcal{A}$ outputs a forgery tuple $(\mathcal{P}^*, \Delta^*, y^*, \sigma^*)$ and wins the security game if the tuple returned by $\mathcal{A}$ is a valid forgery.

The formalization of forward unforgeability follows the definitions of forgeries. We define the forgery by adapting the stronger definition from [FMNP16, CFN18], while our definition handles insider corruption.

Definition 4 (Forgery). Consider the tuple $(\mathcal{P}^* = (f^*, \ell_1^*, \dots, \ell_t^*), \Delta^*, y^*, \sigma^*)$ returned by an adversary $\mathcal{A}$ after executing $\text{Game}_{\mathcal{A}, \text{MKHS}}^{\text{HomUF-CMA}}(\lambda)$, and this is a forgery if $\text{Ver}(\mathcal{P}^*, \Delta^*, \{pk_{id}, ek_{id}\}_{id \in \mathcal{P}^*}, y^*, \sigma^*) = 1$, for all $id \in \mathcal{P}^*$ where we allow $id \in L_{\text{cor}}$ (i.e., all the identities involved in $\mathcal{P}^*$ can be corrupted by $\mathcal{A}$), and either one of the following conditions is satisfied.

Type 1: L_{Δ^*} has not been initialized during the game. That is, the dataset Δ^* was never queried by $\mathcal{A}$.

⁶As indicated in §1.1, this is the same as the scenario where all the signers are honest at the beginning of the game.

Type 2: For all $i \in [t]$, $(\ell_i^*, x_i) \in L_{\Delta^*}$, but $y^* \neq f^*(x_1, \dots, x_t)$. In other words, y^* is not the correct computation result of $\mathcal{P}^*$ executed over signed messages.

Type 3: There exists a label ℓ^* such that $(\ell^*, \cdot) \notin L_{\Delta^*}$. That is, the message with label ℓ^* was never queried by $\mathcal{A}$.

An MKHS scheme is forward unforgeable against insider corruption if no PPT adversary can win the game $\text{Game}_{\mathcal{A}, \text{MKHS}}^{\text{HomUF-CMA}}(\lambda)$ with non-negligible probability.

Remark 1. Catalano et al. [CFN18] proposed two general methods to transform an HS scheme secure against both Type 1 and 2 forgeries into an HS scheme that is also secure against Type 3 forgeries. In Appx. A, we show how to convert our scheme by adopting one of the transformations to achieve security against Type 3 forgeries. Since these transformation methods are also applicable to our scheme, we omit the security analysis for Type 3 forgeries in this work.

4 Construction

4.1 Basic Construction for Single Dataset

Our construction builds on the MKHS scheme due to Fiore et al. [FMNP16] for the evaluation of Boolean circuits of bounded polynomial depth, with the difference that our scheme achieves a stronger security guarantee against signing key leakage. Our construction here follows the syntax in Definition 2, but the evaluation key ek_{id} is generated in conjunction with the signing algorithm in this construction. Since this construction is described in the single-dataset scenario, the dataset identifier Δ is omitted for clarity. This basic construction naturally extends to the multi-dataset scenario by generating new keys for each dataset. Our leveled MKHS scheme HS works as follows:

HS.Setup(1^λ): The admissible class of functions is defined by the circuit with bounded depth $d = d(\lambda) = \text{poly}(\lambda)$.

1. Set $\beta_{max} := 2^{\omega(\log \lambda)^d}$, $\beta_{\text{SIS}} := 2^{\omega(\log \lambda)} \beta_{max}$, then choose an integer $n = \text{poly}(\lambda)$ and a prime $q = O(2^{\text{poly}(\lambda)}) > \beta_{\text{SIS}}$ so that the $\text{SIS}(n, m, q, \beta_{\text{SIS}})$ hardness assumption holds for all $m = \text{poly}(\lambda)$.
2. Let $m^* = m^*(n, q) := O(n \log q)$ and $\beta_{\text{Sam}} := O(n \sqrt{\log q})$ as required by Lemma 2, set $m = \max\{m^*, n \log q + \omega(\log \lambda)\} = \text{poly}(\lambda)$ and $\beta_0 := \beta_{\text{Sam}}$. Let $\text{prms} = (n, m, q, \beta_0, \beta_{max}, \beta_{\text{SIS}})$.
3. Define $\mathcal{U} = \{\mathbf{U} \in \mathbb{Z}_q^{m \times m} : \|\mathbf{U}\|_\infty \leq \beta_{max}\}$, the message space $\mathcal{X} = \{0, 1\}$, the tag space $\mathcal{T} = [T]$, the identity space $\mathcal{ID} \in [ID]$, and the label space $\mathcal{L} = \mathcal{ID} \times \mathcal{T}$, where $T, ID \in \mathbb{N}$.
4. Sample a uniformly random matrix $\mathbf{M} \in \mathbb{Z}_q^{n \times m^7}$.
5. Output the public parameters $\text{pp} = (d, \mathbf{M}, \text{prms}, \mathcal{U}, \mathcal{X}, \mathcal{T}, \mathcal{ID})$.

HS.KeyGen₁(pp): Sample a random matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ along with its trapdoor $\mathbf{T}$ by running $(\mathbf{A}, \mathbf{T}) \leftarrow \text{TrapGen}(1^n, 1^m, q)$ in Lemma 2. Set the signing key $sk = \mathbf{T}$ and public key $pk = \mathbf{A}$. Since the keys can be associated with every identity $id \in \mathcal{ID}$, we denote the output key pair as (sk_{id}, pk_{id}) .

HS.KeyGen₂(pp, ℓ, x): Randomly sample a short $\mathbf{W}_\ell \in \mathcal{U}$ such that $\|\mathbf{W}_\ell\|_\infty \leq \beta_0$. Compute evaluation key $\mathbf{V}_\ell = \mathbf{M}\mathbf{W}_\ell + x \cdot \mathbf{G}$. We note that the collection of $\mathbf{V}_\ell$ represents

⁷ $\mathbf{M}$ works as a common random string. In practice, e.g., this can be generated from the outputs of a hash function with the previously agreed inputs and randomness.

evaluation keys in this construction, and it can be associated with each identity $id \in \mathcal{ID}$. Therefore, in the following, the set of evaluation keys associated with the same id is denoted as ek_{id} .

HS.Sign($sk_{id}, pk_{id}, ek_{id}, \ell, x$): Sample $\mathbf{U}_\ell$ by running **SamPre** in Lemma 2, i.e., $\mathbf{U}_\ell \leftarrow \text{SamPre}(\mathbf{A}_{id}, \mathbf{V}_\ell - x \cdot \mathbf{G}, \mathbf{T}_{id})$. Output the signature $\sigma_\ell := (I, \mathbf{U}_\ell, \mathbf{W}_\ell)$ ⁸ with $\mathbf{U}_\ell, \mathbf{W}_\ell \in \mathcal{U}$ and $I = \{id\} \subseteq \mathcal{ID}$ where I denotes the involved identities.

HS.Eval($f, \{x_i, \sigma_i, ek_i\}_{i \in [t]}$): We treat $f: \mathcal{X}^t \rightarrow \mathcal{X}$ as an arithmetic circuit over $\mathbb{Z}_q$ consisting of fan-in-2 addition and multiplication gates, which can also be used to form all Boolean circuits consisting of **NAND** gates. Let $x_L, \sigma_L := (I_L, \mathbf{U}_L, \mathbf{W}_L), \mathbf{V}_L$ and $x_R, \sigma_R := (I_R, \mathbf{U}_R, \mathbf{W}_R), \mathbf{V}_R$ denote the inputs of the left side and right side of the gate to be evaluated, respectively. The evaluation algorithm works as follows.

1. Set $I = I_L \cup I_R$ and expand $\mathbf{U}_L$ according to the involved identity $id \in I$. For clarity, we write $\mathbf{U}_{\ell=(id, \tau)}$ as $\mathbf{U}_{id}$. Then, expand $\mathbf{U}_L := \{\mathbf{U}_{id}\}_{id \in I_L}$ to $\hat{\mathbf{U}}_L = \{\hat{\mathbf{U}}_{L, id}\}_{id \in I}$ by inserting zero matrices with size $(m \times m)$ as:

$$(\hat{\mathbf{U}}_L \ni) \hat{\mathbf{U}}_{L, id} = \begin{cases} \mathbf{0} & \text{if } id \notin I_L \\ \mathbf{U}_{id} (\in \mathbf{U}_L) & \text{if } id \in I_L \end{cases}, \quad \forall id \in I.$$

The analogous expansion is applied to $\mathbf{U}_R$. Set it as $\hat{\mathbf{U}}_R = \{\hat{\mathbf{U}}_{R, id}\}_{id \in I}$.

2. Define β_L (resp. β_R) as $\|\mathbf{U}_L\|_\infty := \max\{\|\mathbf{U}_{id}\|_\infty : id \in I_L\}$ (resp. $\|\mathbf{U}_R\|_\infty := \max\{\|\mathbf{U}_{id}\|_\infty : id \in I_R\}$). Since $\mathbf{U}$ and $\mathbf{W}$ are evaluated under the same operation, we use the same symbol for evaluating the noise growth.

3. Compute homomorphic signatures gate by gate⁹:

Addition gate: Compute $y = x_L + x_R$, $\mathbf{U} = \{\mathbf{U}_{id}\}_{id \in I} := \{\hat{\mathbf{U}}_{L, id} + \hat{\mathbf{U}}_{R, id}\}_{id \in I}$, $\mathbf{W} = \mathbf{W}_L + \mathbf{W}_R$, and $\mathbf{V} = \mathbf{V}_L + \mathbf{V}_R$. For the noise growth, it holds $\beta := \|\mathbf{U}\|_\infty \leq \beta_L + \beta_R$.

Multiplication gate: Compute $y = x_L \cdot x_R$, $\mathbf{V} = \mathbf{V}_R \mathbf{G}^{-1}(\mathbf{V}_L)$. Set $\mathbf{U} = \{\mathbf{U}_{id}\}_{id \in I} := \{x_R \hat{\mathbf{U}}_{L, id} + \hat{\mathbf{U}}_{R, id} \cdot \mathbf{G}^{-1}(\mathbf{V}_L)\}_{id \in I}$ and $\mathbf{W} = x_R \mathbf{W}_L + \mathbf{W}_R \cdot \mathbf{G}^{-1}(\mathbf{V}_L)$. For the noise growth, it holds $\beta := \|\mathbf{U}\|_\infty \leq |x_R| \beta_L + m \beta_R$.

Addition-with-constant gate: Let the single input be x_L , and we have $y = x_L + a$, where $a \in \mathbb{Z}_q$. Set $\mathbf{U} = \{\mathbf{U}_{id}\}_{id \in I_L}$, $\mathbf{W} = \mathbf{W}_L$, and $\mathbf{V} = \mathbf{V}_L + a \cdot \mathbf{G}$. The noise stays the same.

Multiplication-by-constant gate: Let the single input be x_R , and we have $I = I_R$, $y = a \cdot x_R$, where $a \in \mathbb{Z}_q$. Compute $\mathbf{U} = \{\mathbf{U}_{id}\}_{id \in I} = \{\hat{\mathbf{U}}_{R, id} \cdot \mathbf{G}^{-1}(a \cdot \mathbf{G})\}_{id \in I}$, $\mathbf{W} = \mathbf{W}_R \cdot \mathbf{G}^{-1}(a \cdot \mathbf{G})$, and $\mathbf{V} = \mathbf{V}_R \cdot \mathbf{G}^{-1}(a \cdot \mathbf{G})$. Or alternatively, $\mathbf{U} = \{a \cdot \hat{\mathbf{U}}_{R, id}\}_{id \in I}$, $\mathbf{W} = a \cdot \mathbf{W}_R$, and $\mathbf{V} = a \cdot \mathbf{V}_R$. For the noise growth, it holds $\beta := \|\mathbf{U}\|_\infty \leq m \beta_R$ for the first case, and $\beta := \|\mathbf{U}\|_\infty = |a| \beta_R$ for the second.

4. Output the computation result y and signature $\sigma := (I, \mathbf{U}, \mathbf{W})$.

HS.Ver($\mathcal{P}, \{pk_{id}, ek_{id}\}_{id \in \mathcal{P}}, y, \sigma$): The verification algorithm takes as input the program $\mathcal{P} = (f, \ell_1, \dots, \ell_t)$, a set of involved public and evaluation keys $\{pk_{id}, ek_{id}\}_{id \in \mathcal{P}}$, the computation output y and its corresponding signature $\sigma = (I, \mathbf{U}, \mathbf{W})$, and then proceeds as follows.

⁸We note that when a signer submits to the cloud server the message x with ek_{id} , where $\mathbf{V}_\ell = \mathbf{A}_{id} \mathbf{U}_\ell + x \cdot \mathbf{G} = \mathbf{M} \mathbf{W}_\ell + x \cdot \mathbf{G}$, we assume that $\mathbf{V}_\ell$ and $\mathbf{A}_{id}$ are made public to all the potential verifiers, and that the cloud server cannot change them later as in [GVW15, FMNP16]. Besides, only the cloud server can verify the signing correctness, and the potential verifiers do not receive each input bit and corresponding signature.

⁹The evaluation of $\mathbf{U}$ is conducted component-wise based on the identities, whereas the evaluation of $\mathbf{W}$ is through matrix computations.

1. Check $\mathbf{V}_\ell \neq \mathbf{0}$ and $\mathbf{V}_\ell \neq \mathbf{G}$ for every $\mathbf{V}_\ell \in \{ek_{id}\}_{id \in \mathcal{P}}$. This step ensures all $\mathbf{W}_\ell$'s are not simply set to be $\mathbf{0}$.
2. Compute evaluation key $\mathbf{V}$ corresponding to the computation result y from the circuit f and the input evaluation keys. If we let $\mathbf{V}_L$ and $\mathbf{V}_R$ denote the evaluation keys corresponding to the two inputs of a gate, then for every addition gate, we have $\mathbf{V} = \mathbf{V}_L + \mathbf{V}_R$, and for every multiplication gate, $\mathbf{V} = \mathbf{V}_R \cdot \mathbf{G}^{-1}(\mathbf{V}_L)$ regarding the output of the evaluated gate. Moreover, for every addition-with-constant gate (resp. multiplication-by-constant gate) with evaluation key $\mathbf{V}_L$, we have $\mathbf{V} = \mathbf{V}_L + a \cdot \mathbf{G}$ (resp. $\mathbf{V} = a\mathbf{V}_L$). We denote the computation over evaluation keys as $\mathbf{V} \leftarrow \text{process}_f(\{\mathbf{V}_\ell\})$. We note that $\mathbf{V}$ can be computed in advance before receiving y and signature.
3. Parse σ to obtain $\mathbf{U} = \{\mathbf{U}_{id}\}_{id \in I}$ and $\mathbf{W}$, and then check:

$$I = \{id : id \in \mathcal{P}\} \quad (1)$$

$$\|\mathbf{U}\|_\infty \leq \beta_{max} \quad \text{and} \quad \|\mathbf{W}\|_\infty \leq \beta_{max} \quad (2)$$

$$\sum_{id \in I} \mathbf{A}_{id} \mathbf{U}_{id} + y \cdot \mathbf{G} = \mathbf{V} \bmod q \quad (3)$$

$$\mathbf{M}\mathbf{W} + y \cdot \mathbf{G} = \mathbf{V} \bmod q \quad (4)$$

Roughly speaking, only Eq. (3) is checked in [FMNP16], so even one leaked signing key (i.e., SIS trapdoor) can easily lead to a forgery with a fake result y . To prevent this, we introduced $\mathbf{M}$ whose trapdoor is unknown to signers, which enables our construction to withstand insider corruption.

Signing Correctness. We prove the signing correctness of $\sigma := (I = \{id\}, \mathbf{U}, \mathbf{W})$ for message x with label $\ell := (id, \tau)$. Formally, we prove that $\text{Ver}(I_\ell, pk_{id}, ek_{id}, x, \sigma) = 1$ by checking that conditions from (1) to (4) hold. It is immediate to check (1) because $I = \{id \in I_\ell\}$. Assume that the noise of every fresh signature is bounded by $\beta_0 \leq \beta_{max}$, and then it is evident that (2) holds. For Eq. (3) and (4) we have

$$\begin{aligned} \sum_{id \in I} \mathbf{A}_{id} \mathbf{U}_{id} + x \cdot \mathbf{G} &= \mathbf{A}_{id} \mathbf{U}_{id} + x \cdot \mathbf{G} = \mathbf{V}_\ell = \mathbf{V} \bmod q \quad \text{and} \\ \mathbf{M}\mathbf{W}_\ell + x \cdot \mathbf{G} &= \mathbf{V}_\ell = \mathbf{V} \bmod q \end{aligned}$$

from the definition of the signing algorithm.

Evaluation Correctness. We prove the evaluation correctness for generic fan-in-2 addition and multiplication gates by sequentially proceeding with the checks of the verification algorithm. Let $x_L, \sigma_L := (I_L, \mathbf{U}_L, \mathbf{W}_L), \mathbf{V}_L$ and $x_R, \sigma_R := (I_R, \mathbf{U}_R, \mathbf{W}_R), \mathbf{V}_R$ denote the inputs of the left/right sides of each gate, respectively. Let y and $\sigma := (I, \mathbf{U}, \mathbf{W})$ denote the evaluation result and its signature. Let $I = I_L \cup I_R$, and it is evident that (1) holds. Also, (2) is correct from the analysis of noise growth. We then prove the correctness of (3) and (4) for addition/multiplication gates by induction. Intuitively, assume that the values of the input wires of an intermediate gate satisfy $\mathbf{V}_L = \sum_{id \in I_L} \mathbf{A}_{id} \mathbf{U}_{id} + x_L \mathbf{G}$, $\mathbf{V}_R = \sum_{id \in I_R} \mathbf{A}_{id} \mathbf{U}_{id} + x_R \mathbf{G}$. For an instance of a multiplication gate, we compute $y_k = x_L \cdot x_R$, $\mathbf{U} = \{\mathbf{U}_{id}\}_{id \in I} = \{x_R \cdot \hat{\mathbf{U}}_{L,id} + \hat{\mathbf{U}}_{R,id} \cdot \mathbf{G}^{-1}(\mathbf{V}_L)\}_{id \in I}$, and $\mathbf{V}_{y_k} = \mathbf{V}_R \mathbf{G}^{-1}(\mathbf{V}_L)$.

Thus, we have

$$\begin{aligned}
\sum_{id \in I} \mathbf{A}_{id} \mathbf{U}_{id} + y_k \mathbf{G} &= \sum_{id \in I} \mathbf{A}_{id} (x_R \hat{\mathbf{U}}_{L,id} + \hat{\mathbf{U}}_{R,id} \mathbf{G}^{-1}(\mathbf{V}_L)) + x_L x_R \cdot \mathbf{G} \\
&= x_R \cdot \left(\sum_{id \in I_L} \mathbf{A}_{id} \mathbf{U}_{id} + x_L \mathbf{G} \right) + \left(\sum_{id \in I_R} \mathbf{A}_{id} \mathbf{U}_{id} \right) \mathbf{G}^{-1}(\mathbf{V}_L) \\
&= x_R \cdot \mathbf{G} \mathbf{G}^{-1}(\mathbf{V}_L) + \left(\sum_{id \in I_R} \mathbf{A}_{id} \mathbf{U}_{id} \right) \mathbf{G}^{-1}(\mathbf{V}_L) \\
&= \left(\sum_{id \in I_R} \mathbf{A}_{id} \mathbf{U}_{id} + x_R \cdot \mathbf{G} \right) \mathbf{G}^{-1}(\mathbf{V}_L) \\
&= \mathbf{V}_R \mathbf{G}^{-1}(\mathbf{V}_L) = \mathbf{V}_{y_k} \bmod q.
\end{aligned}$$

We give the full proof of evaluation correctness in Appx. B as it is based on straightforward calculations. By induction, Eq. (3) is correct. Since $\mathbf{W}$ is evaluated similarly as in the single identity setting, the analysis of Eq. (4) is analogous.

Noise Growth and Succinctness. Since any Boolean logic can be expressed only by NAND gates, we give an analysis for the admissible depth- d Boolean circuits composed of NAND gates. As the components $\mathbf{U}$ and $\mathbf{W}$ are generated in a similar way, we give the analysis for $\mathbf{U}$ and apply the result to $\mathbf{W}$ trivially. For every NAND gate, we have $\text{NAND}(x_L, x_R) := 1 - x_L \cdot x_R$. If $\mathbf{U}_L, \mathbf{U}_R$ have noise level less than β , resultant $\mathbf{U}$ will have noise level $\beta^* \leq (m+1)\beta$. Assume that the noise for all inputs $\mathbf{U}_{id}$ is bounded by β_0 , for a depth- d circuit, and then we have $\|\mathbf{U}\|_\infty \leq (m+1)^d \beta_0 \leq 2^{O(\log \lambda)^d} \leq \beta_{max}$. Finally, the size of output signature $\sigma := (I, \mathbf{U}, \mathbf{W})$ for the program involving K signers is at most $(K\lambda + (K+1)\beta_{max}) = O(K \cdot \text{poly}(\lambda))$.

Remark 2. In the single-dataset MKHS scheme [FMNP16], the public matrices $\{\mathbf{V}_\ell\}$ are chosen uniformly at random. In contrast, our scheme computes the public matrices $\{\mathbf{V}_\ell\}$ depending on the messages and a public random matrix $\mathbf{M}$. This design allows us to achieve forward unforgeability even under insider corruption by additionally verifying Eq. (4), although it makes efficient verification in Definition 3 more challenging (which we address in §5). Regarding the signature size, since our scheme introduces only one additional component $\mathbf{W}$ to the final signature σ , the signature size increases at most β_{max} .

4.2 Security Analysis

We prove the forward unforgeability of the proposed multi-key homomorphic signature scheme HS against insider corruption. If an adversary $\mathcal{A}$ produces a forgery in HS with non-negligible probability, we show we can construct a PPT adversary $\mathcal{B}$ that uses $\mathcal{A}$ to solve the $\text{SIS}(n, m, q, \beta_{\text{SIS}})$ problem. Since our basic scheme works for the single dataset setting, Type 1 forgeries do not occur. Furthermore, Type 3 forgeries can be handled using a transformation mentioned in Remark 1.

Theorem 1. *If the $\text{SIS}(n, m, q, \beta_{\text{SIS}})$ hardness assumption holds, our HS is unforgeable under insider corruption.*

Proof. An adversary $\mathcal{A}$ and a challenger $\mathcal{C}$ interacts in the security game $\text{Game}_{\mathcal{A}, \text{MKHS}}^{\text{HomUF-CMA}}(\lambda)$ defined in §3.2 for single-dataset query and $\mathcal{A}$ outputs a forgery $(\mathcal{P}^* = (f^*, \ell_1^*, \dots, \ell_t^*), y^*, \sigma^*)$. $\mathcal{A}$ wins the security game if $\text{Ver}(\mathcal{P}^*, \{pk_{id}, ek_{id}\}_{id \in \mathcal{P}^*}, y^*, \sigma^*) = 1$ and the following condition is satisfied:

Type 2 Forgery: There exist messages $x_{\ell_1^*}, \dots, x_{\ell_t^*}$ such that $y^* \neq f^*(x_{\ell_1^*}, \dots, x_{\ell_t^*})$. In other words, y^* is not the correct evaluation result of $\mathcal{P}^*$.

If $\mathcal{A}$ produces a forgery in the security game, we can construct a PPT $\mathcal{B}$ that solves the $\text{SIS}(n, m, q, \beta_{\text{SIS}})$ problem. Suppose that $\mathcal{B}$ is invoked on a random instance $\mathbf{M}' \in \mathbb{Z}_q^{n \times m}$ of the $\text{SIS}(n, m, q, \beta_{\text{SIS}})$ problem and is asked to output a solution $\mathbf{w} \in \mathbb{Z}^m \setminus \{\mathbf{0}\}$ such that $\mathbf{M}'\mathbf{w} = \mathbf{0} \pmod q$ and $\|\mathbf{w}\|_\infty \leq \beta_{\text{SIS}}$. $\mathcal{B}$ acts as a challenger in the security game of HS with $\mathcal{A}$. In the security game, $\mathcal{B}$ sets $\mathbf{M}'$ as the uniformly random matrix in pp and proceeds with the game.

Once $\mathcal{A}$ outputs a forgery $(\mathcal{P}^* = (f^*, \ell_1^*, \dots, \ell_t^*), y^*, \sigma^*)$, $\mathcal{B}$ parses σ^* to get $\mathbf{W}^*$. Let $\sigma = (I, \mathbf{U}, \mathbf{W})$ be the honest signature¹⁰ generated by HS.Eval algorithm from signatures corresponding to labels $\ell_1^*, \dots, \ell_t^*$ that were given to $\mathcal{A}$. By the correctness of our scheme, we have Eq. (5). According to the definition of a Type 2 forgery, we have Eq. (6).

$$\mathbf{M}'\mathbf{W} + y \cdot \mathbf{G} = \mathbf{V} \pmod q \quad (5)$$

$$\mathbf{M}'\mathbf{W}^* + y^* \cdot \mathbf{G} = \mathbf{V} \pmod q \quad (6)$$

Combining Eq. (5) and (6), we obtain $\mathbf{M}'\mathbf{W}' = y' \cdot \mathbf{G}$, where $\mathbf{W}' = \mathbf{W} - \mathbf{W}^*$ and $y' = y^* - y \neq 0$. Since $\mathbf{W}, \mathbf{W}^* \in \mathcal{U}$ such that $\|\mathbf{W}\|_\infty, \|\mathbf{W}^*\|_\infty \leq \beta_{\text{max}}$, we have $\|\mathbf{W}'\|_\infty \leq 2\beta_{\text{max}}$. Next, $\mathcal{B}$ samples $\mathbf{r} \xleftarrow{\$} \{0, 1\}^m$ and sets $\mathbf{z} = \mathbf{M}'\mathbf{r}$. Then $\mathcal{B}$ computes $\mathbf{r}' = \mathbf{G}^{-1}(y'^{-1} \cdot \mathbf{z})$ so that $\mathbf{r}' \in \{0, 1\}^m$ and $y' \cdot \mathbf{G}\mathbf{r}' = \mathbf{z}$. Therefore, we have

$$\mathbf{M}'(\mathbf{W}'\mathbf{r}' - \mathbf{r}) = \mathbf{M}'\mathbf{W}'\mathbf{r}' - \mathbf{M}'\mathbf{r} = y'\mathbf{G}\mathbf{r}' - \mathbf{M}'\mathbf{r} = \mathbf{z} - \mathbf{z} = \mathbf{0} \pmod q.$$

Set $\mathbf{w} = \mathbf{W}'\mathbf{r}' - \mathbf{r}$, and we have $\mathbf{M}'\mathbf{w} = \mathbf{0}$ and $\|\mathbf{w}\|_\infty \leq (2m+1)\beta_{\text{max}} \leq \beta_{\text{SIS}}$.

Finally, we show that $\mathbf{w} \neq \mathbf{0}$ (i.e., $\mathbf{W}'\mathbf{r}' \neq \mathbf{r}$) holds with overwhelming probability over the random $\mathbf{r}$ by an entropy argument in [GVW15, FMNP16], even for any worst-case choice of $\mathbf{M}', \mathbf{W}', y'$. We note that even if $\mathbf{r}' = \mathbf{G}^{-1}(y'^{-1} \cdot \mathbf{z})$ depends on $\mathbf{z} = \mathbf{M}'\mathbf{r}$, $\mathbf{z}$ is too small to reveal much information about $\mathbf{r}$. Further, since $\mathbf{r}'$ is chosen deterministically based on $\mathbf{z} = \mathbf{M}'\mathbf{r}$, we have $\mathbf{H}_\infty(\mathbf{r}|\mathbf{r}') \geq \mathbf{H}_\infty(\mathbf{r}|\mathbf{M}'\mathbf{r})$. Besides, from Lemma 1, we have $\mathbf{H}_\infty(\mathbf{r}|\mathbf{M}'\mathbf{r}) \geq \mathbf{H}_\infty(\mathbf{r}) - \log(|\mathcal{Z}|)$ where $\mathcal{Z}$ is the space of all possible $\mathbf{z}$. Therefore, we have $\mathbf{H}_\infty(\mathbf{r}|\mathbf{r}') \geq \mathbf{H}_\infty(\mathbf{r}|\mathbf{M}'\mathbf{r}) \geq \log 2^m - \log q^n = \omega(\log \lambda)$. Thus we have

$$\Pr[\mathbf{r} = \mathbf{W}'\mathbf{r}'] \leq 2^{-\mathbf{H}_\infty(\mathbf{r}|\mathbf{r}')} \leq 2^{-\omega(\log \lambda)} = \text{negl}(\lambda).$$

In conclusion, if $\mathcal{A}$ wins the security game, $\mathcal{B}$ can find a valid solution $\mathbf{w}$ to solve the $\text{SIS}(n, m, q, \beta_{\text{SIS}})$ problem with non-negligible probability. $\square$

Remark 3. To achieve resistance against Type 3 forgeries, we can apply the transformation proposed by Catalano et al. [CFN18] to our scheme, as described in Appx. A. After this transformation, the total signature size becomes $(K\lambda + 2(K+1)\beta_{\text{max}})$. In comparison, the scheme by [FMNP16] results in a signature size of $(K\lambda + 2K\beta_{\text{max}})$. Thus, our scheme requires only two extra components of size at most $2\beta_{\text{max}}$ to achieve a stronger level of security compared to [FMNP16].

5 A Variant with Efficient Verification for Multiple Datasets

The scheme in §4.1 achieves stronger security than [FMNP16], but for the multi-dataset scenario, it lacks efficient verification in terms of Definition 3. To mitigate this limitation, we propose a variant MDHS, allowing the reuse of evaluation keys across multiple datasets

¹⁰We define the honest signature as the result computed from the signatures returned by the signing oracle using the previously declared message-tag pairs.

by introducing an uncorrupted helper entity (called proxy server) that collaborates with signers to generate signatures without learning the signed messages. In §5.1, we further reduce the trust required in the proxy server by incorporating multiple proxy servers, ensuring that the scheme remains forward-unforgeable as long as at least one proxy server is honest.

The construction partially follows the intuition of [GVW15, FMNP16]. Signers and the proxy server produce new MKHS keys for each dataset except evaluation keys. Using pseudo-random functions (PRF) enables consistent key generation for the same dataset, eliminating the need to sign the entire dataset at once. Then the public keys and dataset identifier are signed by a standard signature scheme SSig . We use $\text{SSig} = (\text{S.KeyGen}, \text{S.Sign}, \text{S.Ver})$ as a building block to sign each dataset information. For each new dataset Δ , each signer and the proxy server generate fresh keys $(sk_{id}^\Delta, pk_{id}^\Delta), (sk_{ser}^\Delta, pk_{ser}^\Delta)$. The multi-dataset MKHS scheme is given as follows.

MD.Setup(1^λ): Generate public parameters $\text{pp}_H = (d, \text{prms}, \mathcal{U}, \mathcal{V}, \mathcal{X}, \mathcal{T}, \mathcal{ID})$ where $\mathcal{V} = \{\mathbf{V} \in \mathbb{Z}_q^{n \times m}\}$. Then fix a PRF $F: \mathcal{K} \times \{0, 1\}^* \rightarrow \{0, 1\}^k$. Output $\text{pp} = (\text{pp}_H, F, \lambda)$.

MD.SKeyGen(pp): Here $*$ is *ser* or *id*. Randomly select K_* from $\mathcal{K}$ and run $(sk_*^S, vk_*^S) \leftarrow \text{S.KeyGen}(1^\lambda)$ to generate signing-verification key pairs of SSig (for the proxy server or each signer). Set $sk_* = (sk_*^S, K_*)$. We note that $(sk_{ser}, vk_{ser}^S), (sk_{id}, vk_{id}^S)$ will be used as global keys for all the datasets.

MD.KeyGen₁(pp, sk_* , Δ): Here $*$ is *ser* or *id*. Parse sk_* to get K_* and compute the randomness $z_* = F_{K_*}(\Delta)$. Then obtain the signing-public key pairs $(sk_*^\Delta := \mathbf{T}, pk_*^\Delta := \mathbf{A})$ (for the proxy server or each identity) by feeding the randomness to $(\mathbf{A}, \mathbf{T}) \leftarrow \text{TrapGen}(1^n, 1^m, q)$. Output $(sk_{ser}^\Delta, pk_{ser}^\Delta)$ if $*$ = *ser* and $(sk_{id}^\Delta, pk_{id}^\Delta)$ if $*$ = *id*.

MD.KeyGen₂(pp): Randomly select T random matrices $\mathbf{V}_1, \dots, \mathbf{V}_T$ from $\mathcal{V}$ and set evaluation key $ek = (\mathbf{V}_1, \dots, \mathbf{V}_T)$. The set of evaluation keys associated with the same *id* is denoted as ek_{id} , where ek_{id} is signer-specific and reused for all the datasets.

MD.SSig($sk_*^S, pk_*^\Delta, \Delta$): Here $*$ is *ser* or *id*. Run $\sigma_*^\Delta \leftarrow \text{S.Sign}(sk_*^S, pk_*^\Delta \| \Delta)$ to sign the public key pk_*^Δ and dataset identifier Δ by SSig .

MD.Sign(Sign₁($sk_{id}^\Delta, pk_{id}^\Delta, ek_{id}, pk_{ser}^\Delta, \Delta, \ell, x, \sigma_{id}^\Delta$), Sign₂($sk_{ser}^\Delta, pk_{ser}^\Delta, ek_{id}, \Delta, \sigma_{ser}^\Delta$)): The signer with identity $id \in \mathcal{P}$ interacts with the proxy server as follows¹¹.

1. Sign₁ (invoked by the signer) parses ek_{id} to obtain $\mathbf{V}_\ell$ and samples

$$\mathbf{U}_\ell \leftarrow \text{SamPre}(\mathbf{A}_{id}^\Delta, \mathbf{V}_\ell - x \cdot \mathbf{G}, \mathbf{T}_{id}^\Delta)$$

by running SamPre in Lemma 2, and then samples a random short $\mathbf{W}_{\ell, \text{temp}} \in \mathcal{U}$ such that $\|\mathbf{W}_{\ell, \text{temp}}\|_\infty \leq \beta_0$.

2. Sign₁ computes

$$\mathbf{V}_{\ell, \text{temp}} = \mathbf{A}_{ser}^\Delta \cdot \mathbf{W}_{\ell, \text{temp}} + x \cdot \mathbf{G} \bmod q$$

and sends $\mathbf{V}_\ell, \mathbf{V}_{\ell, \text{temp}}$ to Sign₂ (invoked by the proxy server).

3. Sign₂ runs

$$\mathbf{W}_{s, \text{temp}} \leftarrow \text{SamPre}(\mathbf{A}_{ser}^\Delta, \mathbf{V}_\ell - \mathbf{V}_{\ell, \text{temp}}, \mathbf{T}_{ser}^\Delta)$$

and sends $\mathbf{W}_{s, \text{temp}}$ to Sign₁.

¹¹We note that the proxy server does not sign the message associated with each pair of $(\Delta, \mathbf{V}_\ell)$ twice similarly to the signing oracle which also ignores duplicate queries.

4. Sign_1 computes

$$\mathbf{W}_\ell = \mathbf{W}_{s,\text{temp}} + \mathbf{W}_{\ell,\text{temp}} \bmod q.$$

5. Let $\sigma_{\text{HS},\ell}^\Delta = \{I, \mathbf{U}_\ell, \mathbf{W}_\ell\}$ be the signature for the message with label ℓ in dataset Δ , where $I = \{id\} \subset \mathcal{ID}$ denotes the involved identity.

Finally output the signature $\sigma := (I = \{id\}, \sigma_{\text{HS},\ell}^\Delta, \mathbf{pp}_\Delta)$, where $\mathbf{pp}_\Delta$ is defined as $\mathbf{pp}_\Delta = (ek_{id}, pk_{id}^\Delta, \sigma_{id}^\Delta, pk_{ser}^\Delta, \sigma_{ser}^\Delta)$.

MD.Eval($f, \{x_i, \sigma_i, ek_i\}_{i \in [t]}$): Parse $\sigma_i = (I_i, \sigma_{\text{HS},i}^\Delta, \mathbf{pp}_{\Delta,i})$ with $\mathbf{pp}_{\Delta,i} = (\{ek_{id}, pk_{id}^\Delta, \sigma_{id}^\Delta\}_{id \in I_i}, pk_{ser}^\Delta, \sigma_{ser}^\Delta)$ and set $I = \cup_{i=1}^t I_i$, $\mathbf{pp}_\Delta = \cup_{i=1}^t \mathbf{pp}_{\Delta,i}$, $ek_i = \{ek_{id}\}_{id \in I_i}$. Run $\text{HS.Eval}(f, \{x_i, \sigma_{\text{HS},i}^\Delta, ek_i\}_{i \in [t]})$ to obtain the result y with evaluated signature $\sigma_{\text{HS}}^\Delta$ for a function f . Output y with $\sigma = (I, \sigma_{\text{HS}}^\Delta, \mathbf{pp}_\Delta)$.

MD.Ver($\mathcal{P}, \Delta, \{vk_{id}^S\}_{id \in \mathcal{P}}, vk_{ser}^S, y, \sigma$): Parse $\sigma = (I, \sigma_{\text{HS}}^\Delta, \mathbf{pp}_\Delta)$ with $\mathbf{pp}_\Delta = (\{ek_{id}, pk_{id}^\Delta, \sigma_{id}^\Delta\}_{id \in I}, pk_{ser}^\Delta, \sigma_{ser}^\Delta)$. Then for each $id \in I$, check if $\text{S.Ver}(vk_{id}^S, \sigma_{id}^\Delta, pk_{id}^\Delta \| \Delta) = 1$ holds. Moreover, check if $\text{S.Ver}(vk_{ser}^S, \sigma_{ser}^\Delta, pk_{ser}^\Delta \| \Delta) = 1$ holds. If any check fails, the algorithm returns 0. Otherwise, check if $\text{HS.Ver}(\mathcal{P}, pk_{ser}^\Delta, \{pk_{id}^\Delta, ek_{id}\}_{id \in \mathcal{P}}, y, \sigma_{\text{HS}}^\Delta) = 1$ is satisfied. If it does, the algorithm outputs 1, and 0 otherwise.

Correctness. We prove for $\mathcal{P} = \mathcal{I}_\ell$, the component $\mathbf{W}$ of each signature of a message x in Δ is correct. From the definition of the signing algorithm, we have

$$\begin{aligned} \mathbf{A}_{ser}^\Delta \mathbf{W}_\ell + x \cdot \mathbf{G} &= \mathbf{A}_{ser}^\Delta (\mathbf{W}_{s,\text{temp}} + \mathbf{W}_{\ell,\text{temp}}) + x \cdot \mathbf{G} \\ &= \mathbf{V}_\ell - \mathbf{V}_{\ell,\text{temp}} + \mathbf{V}_{\ell,\text{temp}} - x \cdot \mathbf{G} + x \cdot \mathbf{G} = \mathbf{V}_\ell \bmod q. \end{aligned}$$

The evaluation correctness follows from the correctness of SSig , the PRF F , and the basic scheme.

Efficient Verification. Our scheme enables efficient verification in Definition 3. We show below how to construct VerPrep and EffVer .

VerPrep($\mathcal{P}, \{ek_{id}\}_{id \in \mathcal{P}}$): On input a labeled program $\mathcal{P} = (f, \ell_1, \dots, \ell_t)$ and a set of evaluation keys, the algorithm computes $ek_{\mathcal{P}} \leftarrow \text{process}_f(\{ek_{id}\}_{id \in \mathcal{P}})$ (defined in HS.Ver on Page 12). This step is independent of the dataset.

EffVer($ek_{\mathcal{P}}, \Delta, y, \sigma, \{vk_{id}^S\}_{id \in \mathcal{P}}, vk_{ser}^S$): On input $ek_{\mathcal{P}}, \Delta, y$ with its signature σ , and the verification keys of SSig , $\{vk_{id}^S\}_{id \in \mathcal{P}}, vk_{ser}^S$, the algorithm parses $\sigma = (I, \sigma_{\text{HS}}^\Delta, \mathbf{pp}_\Delta)$ with $\mathbf{pp}_\Delta = (\{ek_{id}, pk_{id}^\Delta, \sigma_{id}^\Delta\}_{id \in I}, pk_{ser}^\Delta, \sigma_{ser}^\Delta)$.

Then it checks if $\text{S.Ver}(vk_{id}^S, \sigma_{id}^\Delta, pk_{id}^\Delta \| \Delta) = 1$ holds for every $id \in \mathcal{P}$, and $\text{S.Ver}(vk_{ser}^S, \sigma_{ser}^\Delta, pk_{ser}^\Delta \| \Delta) = 1$ holds. If not, it returns 0. Otherwise it parses $\sigma_{\text{HS}}^\Delta := (I, \mathbf{U}, \mathbf{W})$ to obtain $\mathbf{U} := \{\mathbf{U}_{id}\}_{id \in I}$ and $\mathbf{W}$, and then checks if the conditions (1) to (4) hold utilizing $ek_{\mathcal{P}}$. If all the conditions are satisfied, it outputs 1, and 0 otherwise.

Security. The security of the scheme follows from two main observations: first, the adversary cannot fake the public keys of the single-dataset MKHS scheme because of the security of SSig . Besides, it cannot forge the evaluation result due to the hardness of $\text{SIS}(n, m, q, \beta_{\text{SIS}})$ problem. We slightly modify the security game $\text{Game}_{\mathcal{A}, \text{MKHS}}^{\text{HomUF-CMA}}(\lambda)$ defined in §3.2 to the security game $\text{Game}_{\mathcal{A}, \text{MD-MKHS}}^{\text{HomUF-CMA}}(\lambda)$ by incorporating a proxy server. The modified game proceeds as follows.

Init. The adversary $\mathcal{A}$ declares message-tag pairs $\{(x_\tau, \tau)\}_{\tau \in \mathcal{T}, k \subseteq \mathcal{T}}$ following the rules of $\text{Game}_{\mathcal{A}, \text{MKHS}}^{\text{HomUF-CMA}}(\lambda)$.

Setup. The challenger $\mathcal{C}$ generates public parameter $\mathbf{pp}$ and evaluation key ek , then sends them to $\mathcal{A}$.

Signing Query. $\mathcal{A}$ can adaptively query (id, Δ) and $\mathcal{C}$ responds as in $\text{Game}_{\mathcal{A}, \text{MKHS}}^{\text{HomUF-CMA}}(\lambda)$. Additionally, if this is the first query for the dataset Δ , $\mathcal{C}$ provides $\mathcal{A}$ with the public key of proxy server pk_{ser}^Δ .

Corruption Query. $\mathcal{A}$ and $\mathcal{C}$ interacts as in $\text{Game}_{\mathcal{A}, \text{MKHS}}^{\text{HomUF-CMA}}(\lambda)$. Moreover, $\mathcal{C}$ additionally sends the public key of proxy server pk_{ser}^Δ and the view of corrupted signers for interaction with the proxy servers.

Forgery. $\mathcal{A}$ outputs a forgery (as defined in Definition 4) as in $\text{Game}_{\mathcal{A}, \text{MKHS}}^{\text{HomUF-CMA}}(\lambda)$.

In $\text{Game}_{\mathcal{A}, \text{MD-MKHS}}^{\text{HomUF-CMA}}(\lambda)$, $\mathcal{A}$ is allowed to corrupt all the signers during the corruption query phase, while the proxy server stays uncorrupted.

Theorem 2. *If SSig is a secure signature scheme and $\text{SIS}(n, m, q, \beta_{\text{SIS}})$ hardness assumption holds, the multi-dataset MKHS scheme MDHS is secure.*

We prove Theorem 2 by considering Type 1 and 2 forgeries separately. First, in Lemma 3, we show that if $\mathcal{A}$ outputs a Type 1 forgery, there exists a forger $\mathcal{F}$ that produces an existential forgery against SSig. The security game between $\mathcal{A}$ and a challenger $\mathcal{C}$ is defined in §3.2, and $\mathcal{C}$ additionally sends verification keys of SSig in the setup phase. We note that $\mathcal{C}$ interacts with the external signing oracle of SSig to generate regular signatures of the proxy server.

Lemma 3 (Type 1 Forgery). *If there exists an adversary $\mathcal{A}$ that produces a Type 1 forgery in the security game of MDHS with non-negligible probability, there exists a forger $\mathcal{F}$ that breaks the existential unforgeability of SSig.*

Proof. In the security game, the challenger $\mathcal{C}$ acting as $\mathcal{F}$ is given a verification key from the security game of SSig, and uses it as vk_{ser}^S . Suppose that $\mathcal{A}$ produces a Type 1 forgery, i.e., the dataset Δ^* was never queried during the game. Then $\mathcal{C}$ parses the Type 1 forgery returned by $\mathcal{A}$ to obtain $\sigma^* = (I^*, \sigma_{\text{HS}}^{\Delta^*}, \text{pp}_{\Delta^*})$ with $\text{pp}_{\Delta^*} = (\{ek_{id}, pk_{id}^{\Delta^*}, \sigma_{id}^{\Delta^*}\}_{id \in I^*}, pk_{ser}^{\Delta^*}, \sigma_{ser}^{\Delta^*})$. The signature $\sigma_{ser}^{\Delta^*}$ for the concatenation $(pk_{ser}^{\Delta^*} \parallel \Delta^*)$ is valid although no signature on $(\cdot \parallel \Delta^*)$ was generated by $\mathcal{C}$. This implies an existential forgery against SSig. $\square$

For Type 2 forgeries, we define the following sequence of games and prove that the probability that the adversary produces a Type 2 forgery is negligible.

- **Game 0:** This is the real security game $\text{Game}_{\mathcal{A}, \text{MD-MKHS}}^{\text{HomUF-CMA}}(\lambda)$.
- **Game 1:** This is the same as Game 0 except that all the calls to the PRF of the proxy server are replaced with the calls to the true random function.

Let $G_i(\mathcal{A})$ denote the event that $\mathcal{A}$ wins Game i . To prove the probability that $\mathcal{A}$ generates Type 2 forgeries is negligible, we utilize the game transition from **Game 0** to **Game 1**, and prove that $G_1(\mathcal{A})$ is negligible. We note that $\mathcal{A}$ is not allowed to submit a corruption query for the proxy server.

Lemma 4. *For every PPT adversary $\mathcal{A}$, there exists a PPT distinguisher $\mathcal{D}$ such that $|\Pr[G_0(\mathcal{A})] - \Pr[G_1(\mathcal{A})]| = \text{Adv}_{F, \mathcal{D}}^{\text{PRF}}(\lambda)$ where $\text{Adv}_{F, \mathcal{D}}^{\text{PRF}}(\lambda)$ is the advantage of $\mathcal{D}$ to distinguish the PRF instance $F_{K_{ser}}(\cdot)$ from a truly random function $\mathcal{R}(\cdot)$.*

Proof. $\mathcal{D}$ interacts with $\mathcal{A}$ in the MDHS game and interacts with the external PRF oracle. $\mathcal{D}$ replaces the calls to $F_{K_{ser}}(\cdot)$ with the calls to the PRF oracle. $\mathcal{D}$ outputs 1 in the PRF security game if $\mathcal{A}$ wins the game of MDHS, and $\mathcal{D}$ outputs 0 if $\mathcal{A}$ loses. If a pseudorandom function exists behind the PRF oracle, $\mathcal{D}$ and $\mathcal{A}$ are in Game 0, and if a truly random function exists behind the PRF oracle, $\mathcal{D}$ and $\mathcal{A}$ are in Game 1. Thus we have $\text{Adv}_{F, \mathcal{D}}^{\text{PRF}}(\lambda) = |\Pr[G_0(\mathcal{A})] - \Pr[G_1(\mathcal{A})]|$. $\square$

From Lemma 4, if F is a secure PRF, $\text{Adv}_{F,D}^{\text{PRF}}(\lambda)$ is negligible, so we have if $\Pr[G_1(\mathcal{A})]$ is negligible, $\Pr[G_0(\mathcal{A})]$ is negligible.

Lemma 5 (Type 2 Forgery). *For every PPT $\mathcal{A}$ that produces a Type 2 forgery in the security game of MDHS, there exists a PPT $\mathcal{B}$ such that $\Pr[G_1(\mathcal{A})] \leq Q_\Delta \cdot \text{Adv}_{\mathcal{B}}^{\text{SIS}}(\lambda)$ where $\text{Adv}_{\mathcal{B}}^{\text{SIS}}(\lambda)$ is the advantage of $\mathcal{B}$ breaking the $\text{SIS}(n, m, q, \beta_{\text{SIS}})$ hardness assumption, and Q_Δ is the number of datasets queried by $\mathcal{A}$.*

Proof. Suppose that for an instance $\mathbf{A}' \in \mathbb{Z}_q^{n \times m}$ of $\text{SIS}(n, m, q, \beta_{\text{SIS}})$, $\mathcal{B}$ is asked to output $\mathbf{w} \in \mathbb{Z}^m \setminus \{\mathbf{0}\}$ such that $\mathbf{A}'\mathbf{w} = \mathbf{0} \bmod q$ and $\|\mathbf{w}\|_\infty \leq \beta_{\text{SIS}}$.

Initially, $\mathcal{A}$ declares that Q_{id} identities and Q_Δ datasets will be queried during the game, and sends $\mathcal{B}$ the message-tag pairs $\{x_\tau, \tau\}_{\tau \in \mathcal{T}_{i,k}}$ for each $i \in [Q_{id}]$, $k \in [Q_\Delta]$, where $\mathcal{T}_{i,k} \subseteq \mathcal{T} := \{\tau_1, \dots, \tau_T\}$. $\mathcal{B}$ interacts with $\mathcal{A}$ in Game 1 with the following modified generation of evaluation keys and signatures. $\mathcal{B}$ guesses the dataset index on which $\mathcal{A}$ produces a Type 2 forgery by choosing $d \leftarrow [Q_\Delta]$ at random and sets $\mathbf{A}'$ as the public key of the proxy server. Assume that for the d -th dataset, $\mathcal{A}$ declares message-tag pairs, including D messages. Then $\mathcal{B}$ randomly samples $\mathbf{W}'_1, \dots, \mathbf{W}'_D \leftarrow \mathcal{U}$ and $\mathbf{W}''_1, \dots, \mathbf{W}''_D \leftarrow \mathcal{U}$, and then computes

$$\begin{aligned} \{\mathbf{V}'_j\}_{j \in [D]} &= \{\mathbf{A}' \cdot \mathbf{W}'_j + x_j \cdot \mathbf{G} \bmod q\}_{j \in [D]} \\ \{\mathbf{V}''_j\}_{j \in [D]} &= \{\mathbf{A}' \cdot \mathbf{W}''_j \bmod q\}_{j \in [D]}. \end{aligned}$$

$\mathcal{B}$ sets $ek = (\mathbf{V}'_1 + \mathbf{V}''_1 \bmod q, \dots, \mathbf{V}'_D + \mathbf{V}''_D \bmod q)$ as the evaluation key for all the datasets queried by $\mathcal{A}$.

We note that $\mathcal{B}$ sends $\mathcal{A}$ the matrix $\mathbf{A}'$ as the public key pk_{ser}^Δ of the proxy server for the d -th dataset, and generates keys for the other $(Q_\Delta - 1)$ datasets itself except the evaluation keys. Besides, $\mathcal{B}$ generates the keys for the queried identities itself, enabling efficient handling of signing and corruption queries as it holds the signing key for queried identities. For all declared (x_τ, τ) in the d -th dataset, $\mathcal{B}$ sets $\mathbf{W}_\ell = \mathbf{W}'_\tau + \mathbf{W}''_\tau \bmod q$ as the signature component of the proxy server¹².

The responses for queries basically follow Game 1, except that $\mathcal{B}$ additionally sends $\mathcal{A}$ the view of signers including the interaction with the proxy server when receiving the corruption queries. $\mathcal{B}$ replies with $\mathbf{W}_{\ell, \text{temp}} = \mathbf{W}'_\tau$, $\mathbf{V}_{\ell, \text{temp}} = \mathbf{V}'_\tau$, and $\mathbf{W}_{s, \text{temp}} = \mathbf{W}''_\tau$ as the view of corrupted signers.

Assume that $\mathcal{A}$ outputs a Type 2 forgery $(\mathcal{P}^* = (f^*, \ell_1^*, \dots, \ell_t^*), \Delta^*, y^*, \sigma^*)$. If Δ^* is not the d -th queried dataset, $\mathcal{B}$ aborts the simulation. Otherwise $\mathcal{B}$ parses the forgery returned by $\mathcal{A}$ to obtain $\sigma^* := (I^*, \sigma_{\text{HS}}^{\Delta^*}, \text{pp}_{\Delta^*})$ where $\sigma_{\text{HS}}^{\Delta^*} := (I^*, \mathbf{U}^*, \mathbf{W}^*)$. Let $\sigma_{\text{HS}}^\Delta = (I, \mathbf{U}, \mathbf{W})$ be the honest signature component generated by Eval algorithm from signatures corresponding to labels $\ell_1^*, \dots, \ell_t^*$ that were given to $\mathcal{A}$. By the correctness of our scheme and according to the definition of a Type 2 forgery, we have

$$\mathbf{A}'\mathbf{W} + y \cdot \mathbf{G} = \mathbf{V} \bmod q \quad \text{and} \quad \mathbf{A}'\mathbf{W}^* + y^* \cdot \mathbf{G} = \mathbf{V} \bmod q.$$

Similarly to the analysis of Eq. (5) and (6) in the proof of Theorem 1, $\mathcal{B}$ can find a valid solution $\mathbf{w} = (\mathbf{W} - \mathbf{W}^*)\mathbf{G}^{-1}((y^* - y)^{-1}\mathbf{A}'\mathbf{r}) - \mathbf{r}$ with a randomly sampled $\mathbf{r} \in \{0, 1\}^m$ to solve $\text{SIS}(n, m, q, \beta_{\text{SIS}})$ from the Type 2 forgery produced by $\mathcal{A}$.

In conclusion, if $\mathcal{A}$ succeeds in Game 1, it follows that $\text{Adv}_{\mathcal{B}}^{\text{SIS}}(\lambda) \geq \frac{\Pr[G_1(\mathcal{A})]}{Q_\Delta}$, where $\frac{1}{Q_\Delta}$ represents the probability that $\mathcal{B}$ guesses the index d correctly. Thus $\Pr[G_1(\mathcal{A})] \leq Q_\Delta \cdot \text{Adv}_{\mathcal{B}}^{\text{SIS}}(\lambda)$. Given that $\text{Adv}_{\mathcal{B}}^{\text{SIS}}(\lambda)$ is negligible, we can conclude that $\Pr[G_1(\mathcal{A})]$ is also negligible. $\square$

¹²This way of generating $\mathbf{W}_\ell$ enables $\mathcal{B}$ to simulate the view of the interactive signing algorithm even when the corresponding/requesting signer is corrupted by $\mathcal{A}$ later.

As a result, from Lemma 5, we have $\Pr[G_1(\mathcal{A})]$ is negligible, and with Lemma 4, we have $\Pr[G_0(\mathcal{A})]$ is negligible for Type 2 forgeries.

5.1 Reducing Trust in the Proxy Server

In this section, we propose an extension to further reduce the trust placed in the proxy server by incorporating multiple independent proxy servers. Let P denote the number of proxy servers. This extension differs from the scheme presented in §5 in the following ways.

1. Each proxy server p maintains its own signing-verification key pair for standard signature scheme SSig and signing-public key pair $(sk_{ser,p}^\Delta, pk_{ser,p}^\Delta)$ for MDHS.
2. The signers interact with each proxy server independently and generate signatures for homomorphic evaluation. Therefore, the signature for a message x with label ℓ in dataset Δ is given by

$$\sigma_{\text{HS},\ell}^\Delta = \{I, \mathbf{U}_\ell, \{\mathbf{W}_{\ell,p}\}_{p \in [P]}\}.$$

3. After evaluation, the cloud server outputs the computation result y with the signature

$$\sigma = (I, \sigma_{\text{HS}}^\Delta, \text{pp}_\Delta)$$

where

$$\sigma_{\text{HS}}^\Delta = \{I, \{\mathbf{U}_{id}\}_{id \in I}, \{\mathbf{W}_p\}_{p \in [P]}\}.$$

4. During verification, the verifier additionally checks all the signature components from the proxy servers $\{\mathbf{W}_p\}_{p \in [P]}$ using the corresponding public keys $pk_{ser,p}^\Delta$. Specifically, the verifier checks:

$$\forall p \in [P], pk_{ser,p}^\Delta \mathbf{W}_p + y \cdot \mathbf{G} = \mathbf{V} \bmod q.$$

In this extension, a malicious cloud server may attempt to corrupt a subset of proxy servers to obtain the signing keys. However, as long as one proxy server remains honest, the malicious server cannot generate a valid signature for an incorrect computation result.

More precisely, suppose the malicious server corrupts all signers and $P - 1$ proxy servers, and provides the forgery

$$\sigma_{\text{HS}}^{\Delta*} = \{I, \{\mathbf{U}_{id}^*\}_{id \in I}, \{\mathbf{W}_p^*\}_{p \in [P]}\}.$$

Assume the remaining uncorrupted proxy server holds the signing-public key pair $(sk_{unc}^\Delta, pk_{unc}^\Delta)$. If the malicious server can cause this forgery to pass verification, it must successfully generate $\mathbf{W}_{unc}^*$ such that

$$pk_{unc}^\Delta \mathbf{W}_{unc}^* + y \cdot \mathbf{G} = \mathbf{V} \bmod q$$

without knowing the corresponding signing key sk_{unc}^Δ . If we set an instance $\mathbf{A}' \in \mathbb{Z}_q^{n \times m}$ of $\text{SIS}(n, m, q, \beta_{\text{SIS}})$ as the uncorrupted server's public key, i.e., $pk_{unc}^\Delta = \mathbf{A}'$, then the forgery $\mathbf{W}_{unc}^*$ can be used to generate a solution to the SIS problem as in the proof of Lemma 5, which contradicts its hardness assumption.

Acknowledgements We would like to thank the anonymous reviewers for their insightful comments and suggestions, which helped to improve the presentation of this paper. This work was supported in part by JST SPRING Grant Number JPMJSP2124, JSPS KAKENHI Grant Numbers 23K18459 and 25K03116, and The Telecommunications Advancement Foundation.

References

- [ABF24] Gaspard Anthoine, David Balbás, and Dario Fiore. Fully-succinct multi-key homomorphic signatures from standard assumptions. In *CRYPTO*, pages 317–351. Springer, 2024. doi:[10.1007/978-3-031-68382-4_10](https://doi.org/10.1007/978-3-031-68382-4_10).
- [ALP13] Nuttapong Attrapadung, Benoît Libert, and Thomas Peters. Efficient completely context-hiding quotable and linearly homomorphic signatures. In *PKC*, pages 386–404. Springer, 2013. doi:[10.1007/978-3-642-36362-7_24](https://doi.org/10.1007/978-3-642-36362-7_24).
- [AP11] Joël Alwen and Chris Peikert. Generating shorter bases for hard random lattices. *Theory of Computing Systems*, 48(3):535–553, 2011. doi:[10.1007/s00224-010-9278-3](https://doi.org/10.1007/s00224-010-9278-3).
- [AP19] Diego F Aranha and Elena Pagnin. The simplest multi-key linearly homomorphic signature scheme. In *Latincrypt*, pages 280–300. Springer, 2019. doi:[10.1007/978-3-030-30530-7_14](https://doi.org/10.1007/978-3-030-30530-7_14).
- [BCFL23] David Balbás, Dario Catalano, Dario Fiore, and Russell WF Lai. Chainable functional commitments for unbounded-depth circuits. In *TCC*, pages 363–393. Springer, 2023. doi:[10.1007/978-3-031-48621-0_13](https://doi.org/10.1007/978-3-031-48621-0_13).
- [BFKW09] Dan Boneh, David Freeman, Jonathan Katz, and Brent Waters. Signing a linear subspace: Signature schemes for network coding. In *PKC*, pages 68–87. Springer, 2009. doi:[10.1007/978-3-642-00468-1_5](https://doi.org/10.1007/978-3-642-00468-1_5).
- [BGI14] Elette Boyle, Shafi Goldwasser, and Ioana Ivan. Functional signatures and pseudorandom functions. In *PKC*, pages 501–519. Springer, 2014. doi:[10.1007/978-3-642-54631-0_29](https://doi.org/10.1007/978-3-642-54631-0_29).
- [CFN18] Dario Catalano, Dario Fiore, and Luca Nizzardo. On the security notions for homomorphic signatures. In *ACNS*, pages 183–201. Springer, 2018. doi:[10.1007/978-3-319-93387-0_10](https://doi.org/10.1007/978-3-319-93387-0_10).
- [CFW12] Dario Catalano, Dario Fiore, and Bogdan Warinschi. Efficient network coding signatures in the standard model. In *PKC*, pages 680–696. Springer, 2012. doi:[10.1007/978-3-642-30057-8_40](https://doi.org/10.1007/978-3-642-30057-8_40).
- [CFW14] Dario Catalano, Dario Fiore, and Bogdan Warinschi. Homomorphic signatures with efficient verification for polynomial functions. In *CRYPTO*, pages 371–389. Springer, 2014. doi:[10.1007/978-3-662-44371-2_21](https://doi.org/10.1007/978-3-662-44371-2_21).
- [DORS08] Yevgeniy Dodis, Rafail Ostrovsky, Leonid Reyzin, and Adam Smith. Fuzzy extractors: How to generate strong keys from biometrics and other noisy data. *SIAM journal on computing*, 38(1):97–139, 2008. doi:[10.1137/060651380](https://doi.org/10.1137/060651380).
- [FMNP16] Dario Fiore, Aikaterini Mitrokotsa, Luca Nizzardo, and Elena Pagnin. Multi-key homomorphic authenticators. In *Asiacrypt*, pages 499–530. Springer, 2016. doi:[10.1007/978-3-662-53890-6_17](https://doi.org/10.1007/978-3-662-53890-6_17).
- [FN16] Dario Fiore and Anca Nitulescu. On the (in)security of SNARKs in the presence of oracles. In *TCC*, pages 108–138. Springer, 2016. doi:[10.1007/978-3-662-53641-4_5](https://doi.org/10.1007/978-3-662-53641-4_5).
- [GPV08] Craig Gentry, Chris Peikert, and Vinod Vaikuntanathan. Trapdoors for hard lattices and new cryptographic constructions. In *STOC*, pages 197–206. ACM, 2008. doi:[10.1145/1374376.1374407](https://doi.org/10.1145/1374376.1374407).

- [GVW15] Sergey Gorbunov, Vinod Vaikuntanathan, and Daniel Wichs. Leveled fully homomorphic signatures from standard lattices. In *STOC*, pages 469–477. ACM, 2015. doi:[10.1145/2746539.2746576](https://doi.org/10.1145/2746539.2746576).
- [GW11] Craig Gentry and Daniel Wichs. Separating succinct non-interactive arguments from all falsifiable assumptions. In *STOC*, pages 99–108. ACM, 2011. doi:[10.1145/1993636.1993651](https://doi.org/10.1145/1993636.1993651).
- [GW13] Rosario Gennaro and Daniel Wichs. Fully homomorphic message authenticators. In *Asiacrypt*, pages 301–320. Springer, 2013. doi:[10.1007/978-3-642-42045-0_16](https://doi.org/10.1007/978-3-642-42045-0_16).
- [JMSW02] Robert Johnson, David Molnar, Dawn Song, and David Wagner. Homomorphic signature schemes. In *CT-RSA*, pages 244–262. Springer, 2002. doi:[10.1007/3-540-45760-7_17](https://doi.org/10.1007/3-540-45760-7_17).
- [LTWC18] Russell WF Lai, Raymond KH Tai, Harry WH Wong, and Sherman SM Chow. Multi-key homomorphic signatures unforgeable under insider corruption. In *Asiacrypt*, pages 465–492. Springer, 2018. doi:[10.1007/978-3-030-03329-3_16](https://doi.org/10.1007/978-3-030-03329-3_16).
- [MP12] Daniele Micciancio and Chris Peikert. Trapdoors for lattices: Simpler, tighter, faster, smaller. In *Eurocrypt*, pages 700–718. Springer, 2012. doi:[10.1007/978-3-642-29011-4_41](https://doi.org/10.1007/978-3-642-29011-4_41).
- [MR04] Daniele Micciancio and Oded Regev. Worst-case to average-case reductions based on gaussian measures. In *FOCS*, pages 372–381. IEEE, 2004. doi:[10.1109/FOCS.2004.72](https://doi.org/10.1109/FOCS.2004.72).
- [PPS21] Chris Peikert, Zachary Pepin, and Chad Sharp. Vector and functional commitments from lattices. In *TCC*, pages 480–511. Springer, 2021. doi:[10.1007/978-3-030-90456-2_16](https://doi.org/10.1007/978-3-030-90456-2_16).
- [SBB19] Lucas Schabhüser, Denis Butin, and Johannes Buchmann. Context hiding multi-key linearly homomorphic authenticators. In *CT-RSA*, pages 493–513. Springer, 2019. doi:[10.1007/978-3-030-12612-4_25](https://doi.org/10.1007/978-3-030-12612-4_25).

A Realizing Stronger Security

Catalano et al. [CFN18] summarized three security notions for homomorphic signature schemes: semi-adaptive, adaptive, and strong adaptive, respectively. Informally, the forgery in the semi-adaptive security game satisfies either one type of the following: (1) the dataset Δ^* was never queried during the game, or (2) the claimed output message y^* is not the correct computation result of the program $\mathcal{P}^*$. These two types of forgery correspond to Type 1 and Type 2 in Definition 4. However, it is stressed that all the inputs of the labeled program should have been queried if the adversaries produce Type 2 forgeries. The strong-adaptive definition addressed the issue that the adversaries may claim the labeled programs in which some inputs are missing in the forgery stage, which is the Type 3 forgery in Definition 4.

Additionally, Catalano et al. [CFN18] proposed a generic transformation to allow a semi-adaptive secure HS scheme to satisfy stronger adaptive security. The main idea of the transformation is to combine a semi-adaptive secure HS scheme Σ with another semi-adaptive secure HS scheme Σ_{OR} that supports only OR operation. Since our scheme supports the evaluation of Boolean circuits, the scheme Σ_{OR} can be instantiated by our leveled MKHS scheme HS.

Let $\text{HS} := (\text{HS.Setup}, \text{HS.KeyGen}, \text{HS.Sign}, \text{HS.Eval}, \text{HS.Ver})$ and $\text{HS}_{\text{OR}} := (\text{HS}_{\text{OR}}.\text{Setup}, \text{HS}_{\text{OR}}.\text{KeyGen}, \text{HS}_{\text{OR}}.\text{Sign}, \text{HS}_{\text{OR}}.\text{Eval}, \text{HS}_{\text{OR}}.\text{Ver})$ denote the semi-adaptive secure HS schemes, in which HS_{OR} supports the circuits that consist of only OR gates. We briefly introduce the transformation to a strong adaptive secure scheme HS' in a black box way.

$\text{HS}'.\text{Setup}(1^\lambda)$: Run the setup algorithms $\text{HS.Setup}(1^\lambda)$ to obtain the public parameters pp .

$\text{HS}'.\text{KeyGen}(\text{pp})$: Run the algorithms $(sk, pk) \leftarrow \text{HS.KeyGen}(\text{pp})$ and $(sk_{\text{OR}}, pk_{\text{OR}}) \leftarrow \text{HS}_{\text{OR}}.\text{KeyGen}(\text{pp})$, and output $sk' := (sk, sk_{\text{OR}})$, $pk' := (pk, pk_{\text{OR}})$.

$\text{HS}'.\text{Sign}(\text{pp}', sk', pk', \ell, x)$: Run the algorithms $\text{HS.Sign}(\text{pp}, sk, pk, \ell, x)$, $\text{HS}_{\text{OR}}.\text{Sign}(\text{pp}, sk_{\text{OR}}, pk_{\text{OR}}, \ell, 0)$ to generate the evaluation key $ek' := (ek, ek_{\text{OR}})$ and the signature $\sigma' := (\sigma, \sigma_{\text{OR}})$. We note that $\text{HS}_{\text{OR}}.\text{Sign}$ signs the bit 0 instead of message x .

$\text{HS}'.\text{Eval}(f, \{x_i, \sigma'_i, ek'_i\}_{i \in [t]})$: Run $\sigma_{\text{out}} \leftarrow \text{HS.Eval}(f, \{x_i, \sigma_i, ek_i\}_{i \in [t]})$ and $\sigma_{\text{OR}, \text{out}} \leftarrow \text{HS}_{\text{OR}}.\text{Eval}(f_{\text{OR}}, \{0, \sigma_{\text{OR}, i}, ek_{\text{OR}, i}\}_{i \in [t]})$ to get the evaluated signature $\sigma'_{\text{out}} := (\sigma_{\text{out}}, \sigma_{\text{OR}, \text{out}})$. We note that f_{OR} is obtained by replacing all the gates in circuit f with OR gates.

$\text{HS}'.\text{Ver}(\mathcal{P}, pk', ek', y, \sigma')$: Parse $\mathcal{P} = (f, \ell_1, \dots, \ell_t)$, $pk' = (pk, pk_{\text{OR}})$, $ek' = (ek, ek_{\text{OR}})$, and $\sigma' = (\sigma, \sigma_{\text{OR}})$. Then define $\mathcal{P}_{\text{OR}} = (f_{\text{OR}}, \ell_1, \dots, \ell_t)$. After that, check whether $\text{HS.Ver}(\mathcal{P}, pk, ek, y, \sigma) = 1$ and $\text{HS}_{\text{OR}}.\text{Ver}(\mathcal{P}_{\text{OR}}, pk_{\text{OR}}, ek_{\text{OR}}, 0, \sigma_{\text{OR}}) = 1$ where y is the evaluation result.

Catalano et al. [CFN18] proposed and proved the following theorem to show that the transformed scheme is strong-adaptive secure, and in other words, it is unforgeable against Type 3 forgeries.

Theorem 3. *Assume that HS is a semi-adaptive homomorphic signature scheme for a class of Boolean circuits C , and HS_{OR} is a semi-adaptive homomorphic signature scheme supporting OR circuits. Then the scheme HS' described above is a strong-adaptive secure homomorphic signature scheme for the circuit C .*

Therefore, our scheme can also be secure even if the adversaries produce Type 3 forgeries in the security game by applying the transformation instantiated by the proposed scheme.

B Proof of Evaluation Correctness

Let the inputs for the first gate be $x_L, x_R, \sigma_L, \sigma_R, \mathbf{V}_L$, and $\mathbf{V}_R$, where $\mathbf{V}_L = \mathbf{A}_{L, id} \mathbf{U}_{L, id} + x_L \cdot \mathbf{G}$ and $\mathbf{V}_R = \mathbf{A}_{R, id} \mathbf{U}_{R, id} + x_R \cdot \mathbf{G}$.

1. For the first gate, if it is an addition gate, we compute $y_1 = x_L + x_R$, $\mathbf{U} = \{\mathbf{U}_{id}\}_{id \in I} = \{\hat{\mathbf{U}}_{L, id} + \hat{\mathbf{U}}_{R, id}\}_{id \in I}$, and $\mathbf{V}_{y_1} = \mathbf{V}_L + \mathbf{V}_R$. Hence we have

$$\begin{aligned} \sum_{id \in I} \mathbf{A}_{id} \mathbf{U}_{id} + y_1 \mathbf{G} &= \sum_{id \in I} \mathbf{A}_{id} (\hat{\mathbf{U}}_{L, id} + \hat{\mathbf{U}}_{R, id}) + (x_L + x_R) \cdot \mathbf{G} \\ &= \mathbf{A}_{L, id} \mathbf{U}_{L, id} + x_L \cdot \mathbf{G} + \mathbf{A}_{R, id} \mathbf{U}_{R, id} + x_R \cdot \mathbf{G} \\ &= \mathbf{V}_L + \mathbf{V}_R = \mathbf{V}_{y_1} \pmod{q}. \end{aligned}$$

If it is a multiplication gate, we compute $y_1 = x_L \cdot x_R$, $\mathbf{U} = \{\mathbf{U}_{id}\}_{id \in I} := \{x_R \hat{\mathbf{U}}_{L,id} + \hat{\mathbf{U}}_{R,id} \cdot \mathbf{G}^{-1}(\mathbf{V}_L)\}_{id \in I}$, and $\mathbf{V}_{y_1} = \mathbf{V}_R \mathbf{G}^{-1}(\mathbf{V}_L)$. Hence we have

$$\begin{aligned} \sum_{id \in I} \mathbf{A}_{id} \mathbf{U}_{id} + y_1 \mathbf{G} &= \sum_{id \in I} \mathbf{A}_{id} (x_R \hat{\mathbf{U}}_{L,id} + \hat{\mathbf{U}}_{R,id} \mathbf{G}^{-1}(\mathbf{V}_L)) + x_L x_R \cdot \mathbf{G} \\ &= x_R \cdot (\mathbf{A}_{L,id} \mathbf{U}_{L,id} + x_L \mathbf{G}) + \left(\sum_{id \in I_R} \mathbf{A}_{id} \mathbf{U}_{id} \right) \mathbf{G}^{-1}(\mathbf{V}_L) \\ &= x_R \cdot \mathbf{V}_L + (\mathbf{A}_{R,id} \mathbf{U}_{R,id}) \mathbf{G}^{-1}(\mathbf{V}_L) \\ &= x_R \cdot \mathbf{G} \mathbf{G}^{-1}(\mathbf{V}_L) + (\mathbf{A}_{R,id} \mathbf{U}_{R,id}) \mathbf{G}^{-1}(\mathbf{V}_L) \\ &= (\mathbf{A}_{R,id} \mathbf{U}_{R,id} + x_R \cdot \mathbf{G}) \mathbf{G}^{-1}(\mathbf{V}_L) = \mathbf{V}_R \mathbf{G}^{-1}(\mathbf{V}_L) = \mathbf{V}_{y_1} \bmod q. \end{aligned}$$

Thus we have $\sum_{id \in I} \mathbf{A}_{id} \mathbf{U}_{id} + y_1 \mathbf{G} = \mathbf{V}_{y_1}$.

2. Assume that the values of the input wires of $(k+1)$ -th gate satisfy the following equations:

$$\mathbf{V}_L = \sum_{id \in I_L} \mathbf{A}_{id} \mathbf{U}_{id} + x_L \mathbf{G} \bmod q, \quad \mathbf{V}_R = \sum_{id \in I_R} \mathbf{A}_{id} \mathbf{U}_{id} + x_R \mathbf{G} \bmod q.$$

If the $(k+1)$ -th gate is an addition gate, we compute $y_{k+1} = x_L + x_R$, $\mathbf{U} = \{\mathbf{U}_{id}\}_{id \in I} = \{\hat{\mathbf{U}}_{L,id} + \hat{\mathbf{U}}_{R,id}\}_{id \in I}$, and $\mathbf{V}_{y_{k+1}} = \mathbf{V}_L + \mathbf{V}_R$. Thus we have

$$\begin{aligned} \sum_{id \in I} \mathbf{A}_{id} \mathbf{U}_{id} + y_{k+1} \mathbf{G} &= \sum_{id \in I} \mathbf{A}_{id} \mathbf{U}_{id} + (x_L + x_R) \cdot \mathbf{G} \\ &= \sum_{id \in I_L} \mathbf{A}_{id} \mathbf{U}_{id} + x_L \mathbf{G} + \sum_{id \in I_R} \mathbf{A}_{id} \mathbf{U}_{id} + x_R \mathbf{G} \\ &= \mathbf{V}_L + \mathbf{V}_R = \mathbf{V}_{y_{k+1}} \bmod q. \end{aligned}$$

If the $(k+1)$ -th gate is a multiplication gate, we compute $y_{k+1} = x_L \cdot x_R$, $\mathbf{U} = \{\mathbf{U}_{id}\}_{id \in I} = \{x_R \cdot \hat{\mathbf{U}}_{L,id} + \hat{\mathbf{U}}_{R,id} \cdot \mathbf{G}^{-1}(\mathbf{V}_L)\}_{id \in I}$, and $\mathbf{V}_{y_{k+1}} = \mathbf{V}_R \mathbf{G}^{-1}(\mathbf{V}_L)$. Thus, we have

$$\begin{aligned} \sum_{id \in I} \mathbf{A}_{id} \mathbf{U}_{id} + y_{k+1} \mathbf{G} &= \sum_{id \in I} \mathbf{A}_{id} (x_R \hat{\mathbf{U}}_{L,id} + \hat{\mathbf{U}}_{R,id} \mathbf{G}^{-1}(\mathbf{V}_L)) + x_L x_R \cdot \mathbf{G} \\ &= x_R \cdot \left(\sum_{id \in I_L} \mathbf{A}_{id} \mathbf{U}_{id} + x_L \mathbf{G} \right) + \left(\sum_{id \in I_R} \mathbf{A}_{id} \mathbf{U}_{id} \right) \mathbf{G}^{-1}(\mathbf{V}_L) \\ &= x_R \cdot \mathbf{G} \mathbf{G}^{-1}(\mathbf{V}_L) + \left(\sum_{id \in I_R} \mathbf{A}_{id} \mathbf{U}_{id} \right) \mathbf{G}^{-1}(\mathbf{V}_L) \\ &= \left(\sum_{id \in I_R} \mathbf{A}_{id} \mathbf{U}_{id} + x_R \cdot \mathbf{G} \right) \mathbf{G}^{-1}(\mathbf{V}_L) = \mathbf{V}_{y_{k+1}} \bmod q. \end{aligned}$$

Therefore, $\sum_{id \in I} \mathbf{A}_{id} \mathbf{U}_{id} + y_{k+1} \mathbf{G} = \mathbf{V}_{y_{k+1}}$ is correct. By induction, we have shown that Eq. (3) is correct. Since $\mathbf{W}$ is evaluated similarly as in the single identity setting, the analysis of Eq. (4) is analogous.